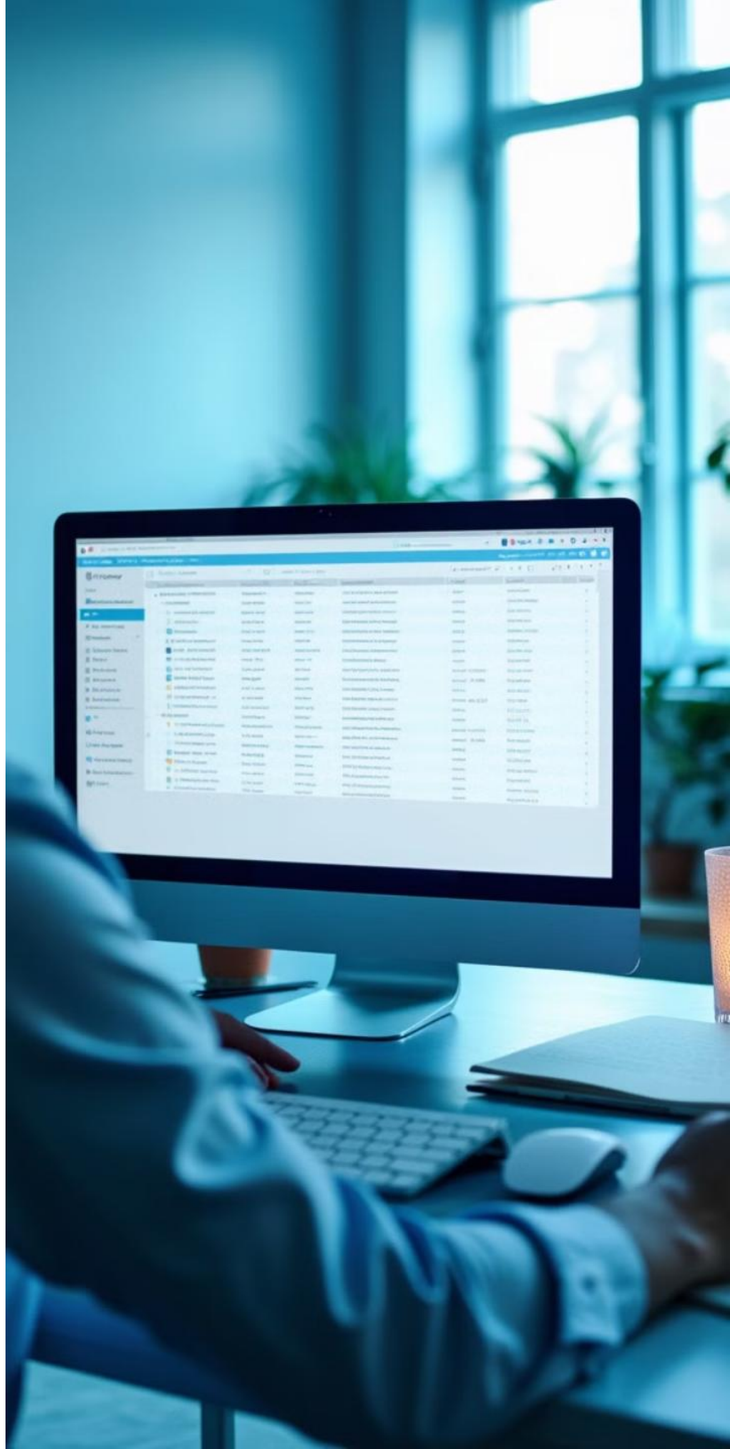
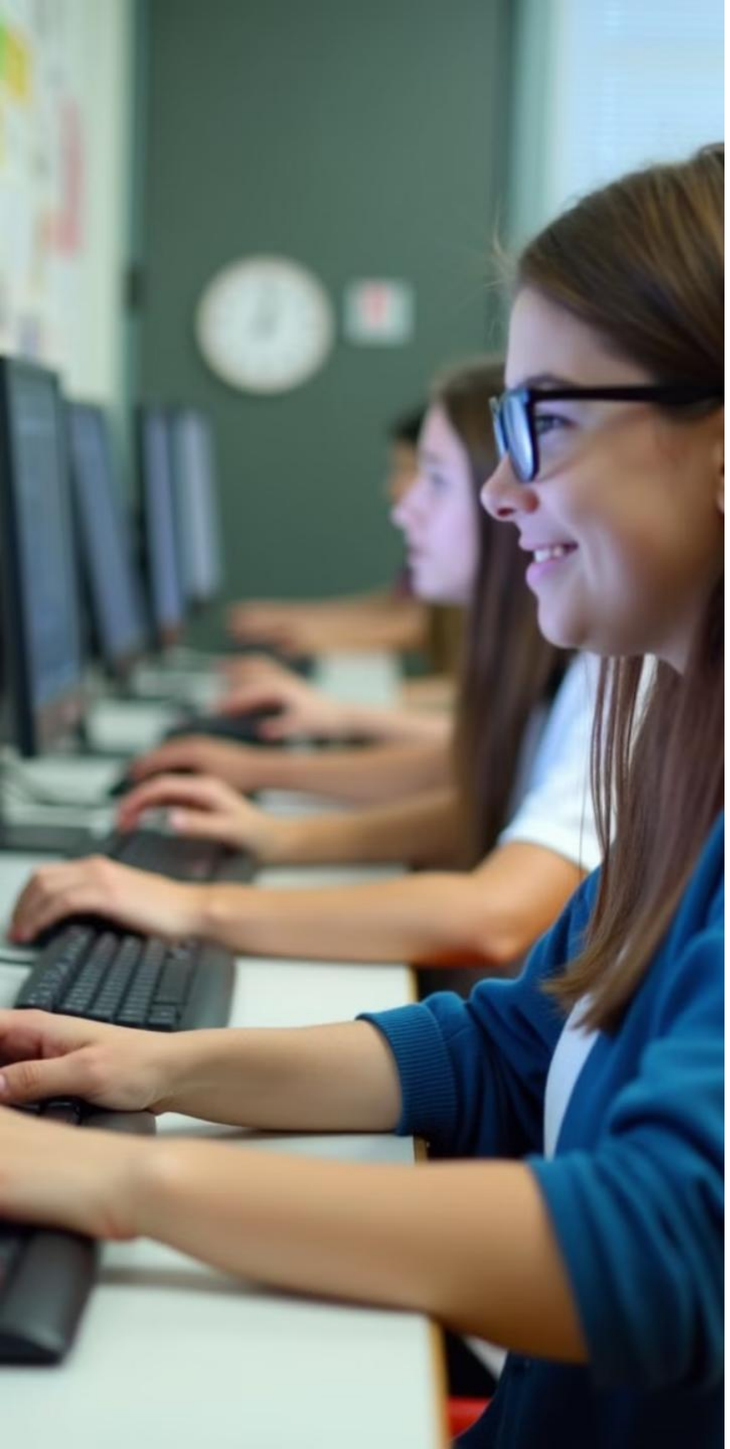
MÓDULO:
ACCESO A DATOS



UT2: Manejo de conectores

En esta unidad, aprenderemos sobre la gestión de información almacenada en bases de datos relacionales mediante conectores.





RESULTADO DE APRENDIZAJE: RA2		
RA2. Desarrolla aplicaciones que gestionan información almacenada en bases de datos relacionales identificando y utilizando mecanismos de conexión.		
CRITERIOS DE EVALUACIÓN		%
a)	Se han valorado las ventajas e inconvenientes de utilizar conectores.	**5%**
b)	Se han utilizado gestores de bases de datos embebidos e independientes.	**10%**
c)	Se ha utilizado el conector idóneo en la aplicación.	**10%**
d)	Se ha establecido la conexión.	**10%**
e)	Se ha definido la estructura de la base de datos.	**10%**
f)	Se han desarrollado aplicaciones que modifican el contenido de la base de datos.	**10%**
g)	Se han definido los objetos destinados a almacenar el resultado de las consultas.	**5%**
h)	Se han desarrollado aplicaciones que efectúan consultas.	**15%**
i)	Se han eliminado los objetos una vez finalizada su función.	**5%**
j)	Se han gestionado las transacciones.	**10%**
k)	Se han ejecutado procedimientos almacenados en la base de datos.	**10%**



Contenidos de la UT2

1. **Introducción**
   Conceptos previos. BBDD relacionales.
2. **Desfase objeto relacional**
   Origen del desfase y soluciones.
3. **Protocolos de acceso a bases de datos: CONECTORES**
   Conectores (ODBC y JDBC).
   Arquitecturas, components, tipos y elección del conector.
4. **Establecimiento de conexiones**
   Conexión desde Java a gestores independientes (MySQL) y embebidos (SQLite)
5. **Operaciones básicas**
   Tipos de operaciones sobre bases de datos relacionales con JDBC



Contenidos de la UT2

6 **Resultado de una consulta**

Manejo, utilización de los resultados de una consulta.
Modificación de datos

7 **Procedimientos almacenados en base de datos**

Ejecución de procedimientos almacenados en BBDD.
Clase CallableStatement

8 **Gestión de transacciones**

Implementación e importancia de las transacciones

UT2 · MANEJO DE CONECTORES

1 · INTRODUCCIÓN



- RA2 Y CRITERIOS DE EVALUACIÓN
- CONCEPTOS PREVIOS

2 · DESFASE OBJETO-RELACIONAL



- ORIGEN
- SOLUCIONES

3 · PROTOCOLOS Y CONECTORES



- ODBC Y JDBC
- ARQUITECTURA, COMPONENTES Y TIPOS
- VENTAJAS E INCONVENIENTES

4 · ESTABLECIMIENTO DE CONEXIONES



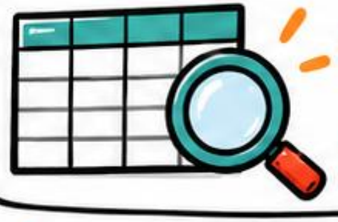
- MYSQL CON DOCKER
- DRIVER Y PROYECTO MAVEN
- APERTURA DE CONEXIONES
- GESTORES EMBEBIDOS E INDEPENDIENTES
- REALIZACIÓN DE CONSULTAS

5 · OPERACIONES BÁSICAS



- STATEMENT
- DDL: ESTRUCTURA DE DATOS
- CONSULTAS Y SENTENCIAS PREPARADAS
- DML: MODIFICACIÓN DE DATOS

6 · RESULTADOS DE CONSULTAS



- TIPOS Y CONFIGURACIÓN DE RESULTSET
- RESULTSET ACTUALIZABLE

7 · PROCEDIMIENTOS ALMACENADOS



- CALLABLESTATEMENT

8 · GESTIÓN DE TRANSACCIONES



- ACID
- AUTO-COMMIT
- COMMIT Y ROLLBACK

9 · BIBLIOGRAFÍA





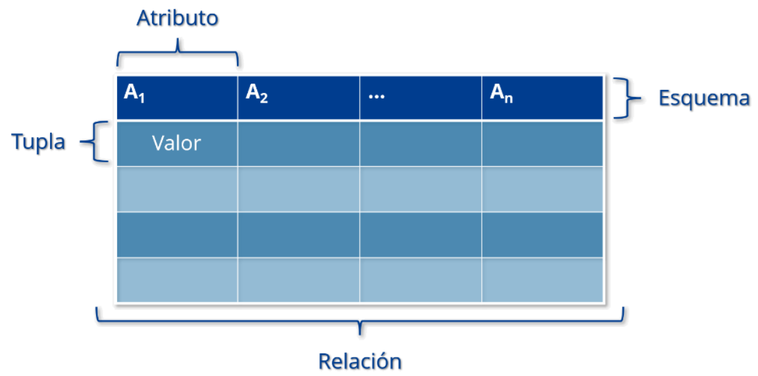


Video introducción (Aula Virtual)

1. Introducción

Conceptos previos

- **Bases de datos relacionales:** Uno de los sistemas de almacenamiento más ampliamente utilizados actualmente
- **Estructura:** Información organizada en tablas compuestas de filas y columnas
- **Filas o tuplas:** Representan un objeto de datos único dentro de la tabla
- **Identificación:** Cada fila se identifica de manera inequívoca mediante una clave primaria
- **Ventajas:** Esta estructura permite que los datos puedan consultarse y relacionarse entre sí de manera eficiente





Conceptos previos

¿Cómo se gestionan las bases de datos?

Para gestionar las bases de datos, se utilizan sistemas:
Database Management Systems (DBMS)

Se encargan de almacenar, mantener y recuperar la información.

En bases de datos relacionales:
Relational Database Management System (RDBMS)

Gran variedad de implementaciones de RDBMS en el mercado -> características y lenguajes de acceso propios.

COMPLICAN LA INTEGRACIÓN DE APLICACIONES



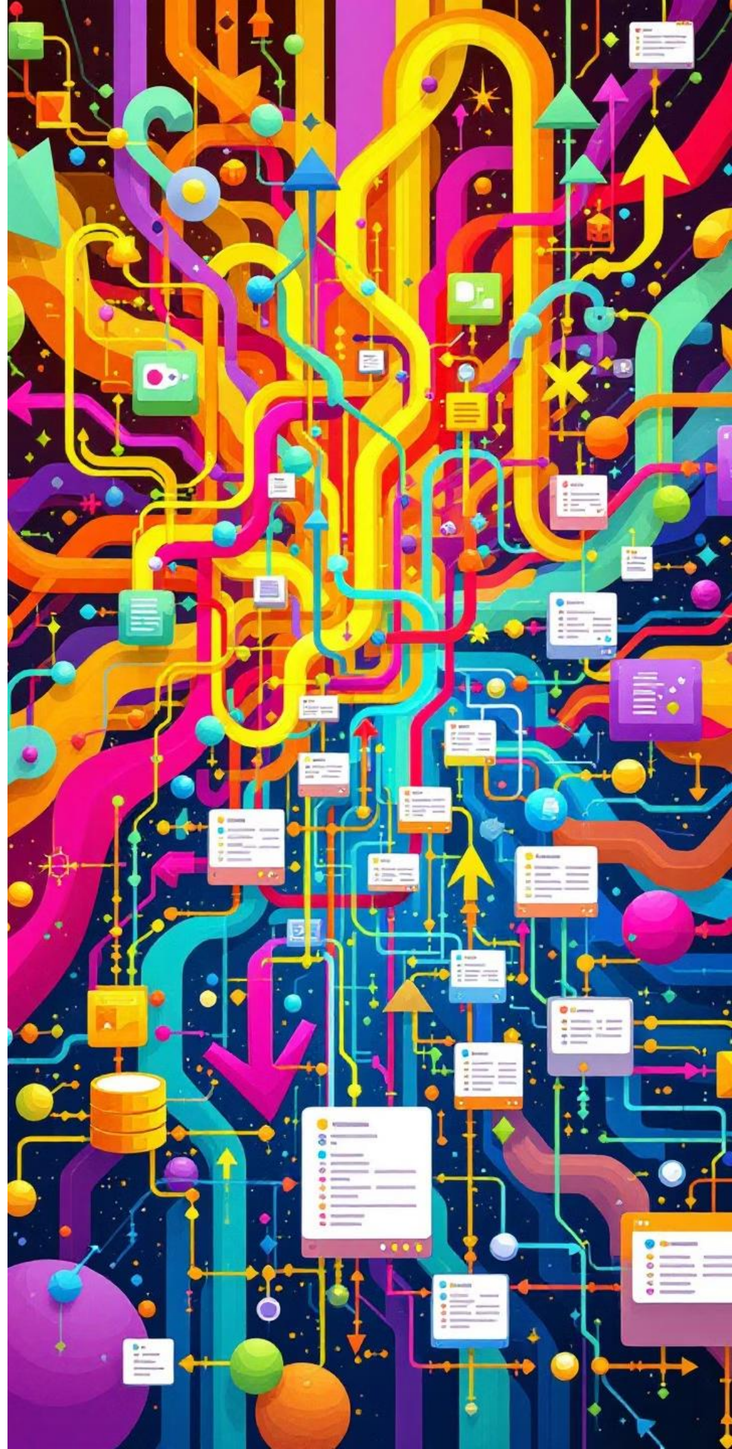
2. Desfase Objeto-Relacional

Origen del desfase

Se refiere a las dificultades técnicas que surgen al **combinar el uso de bases de datos relacionales con** aplicaciones desarrolladas en lenguajes de **programación orientada a objetos (POO)**.

Este desfase se presenta debido a las diferencias fundamentales en el enfoque de ambos paradigmas, y puede observarse en los siguientes aspectos:

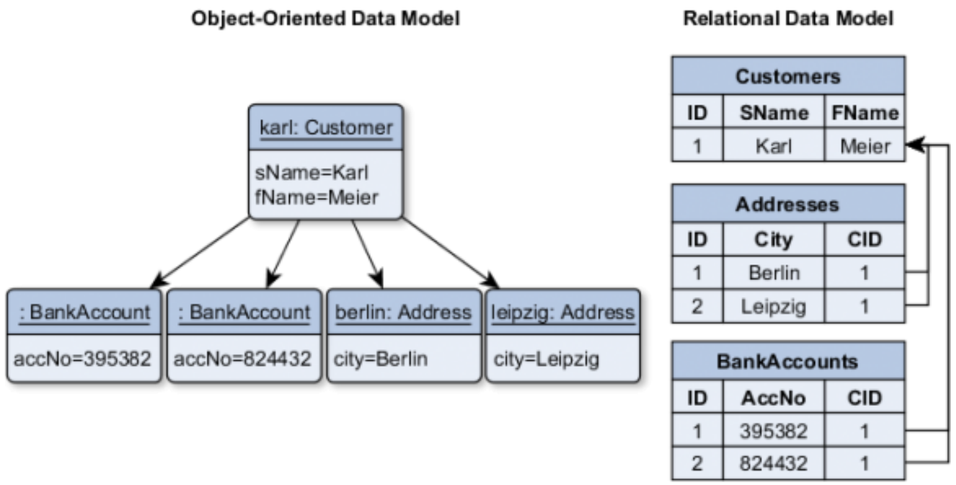
i. Diferencias en el lenguaje y paradigma
ii. Restricciones en los tipos de datos
iii. Traducción entre modelos

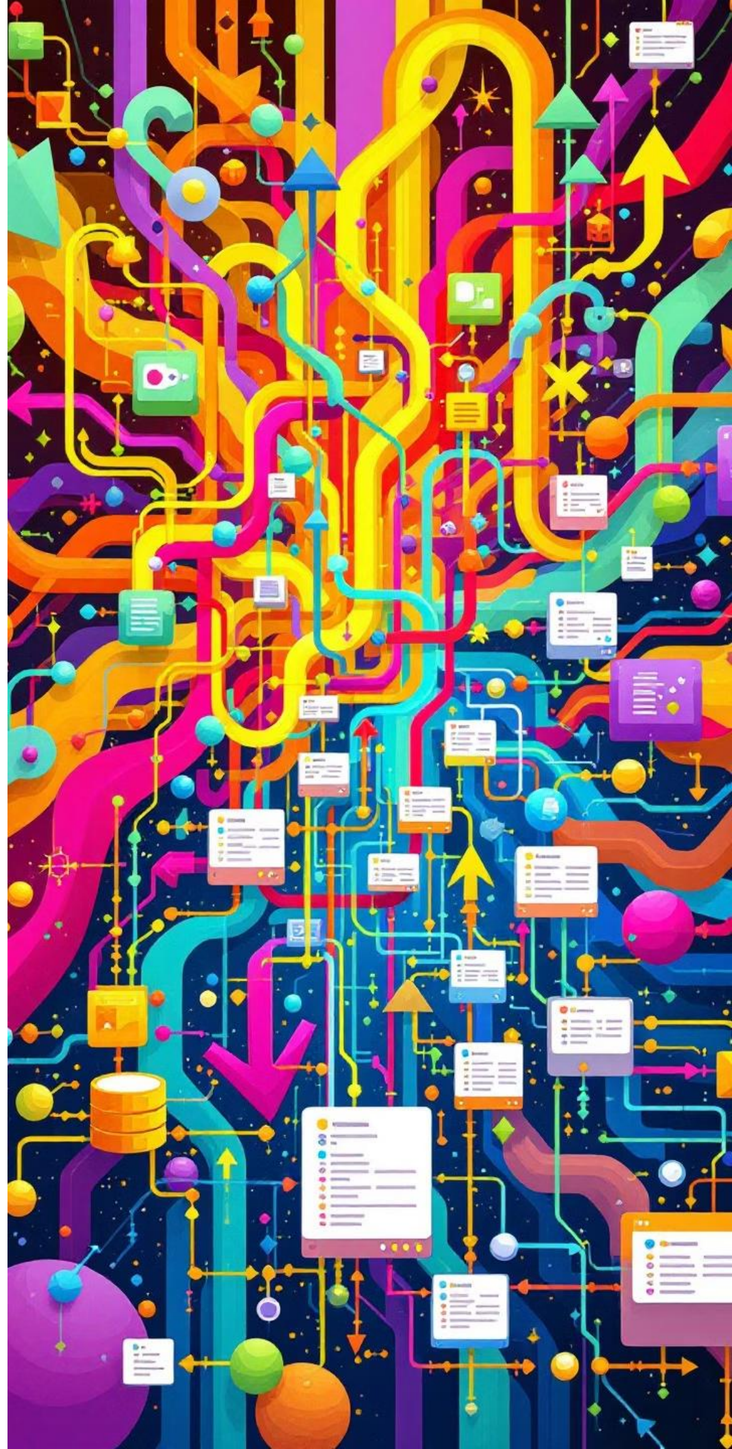


2. Desfase Objeto-Relacional

i. Diferencias en el lenguaje y paradigma

Bases de datos relacionales	→ Modelo basado en tablas
Programación orientada a objetos	→ Datos estructurados en objetos y clases

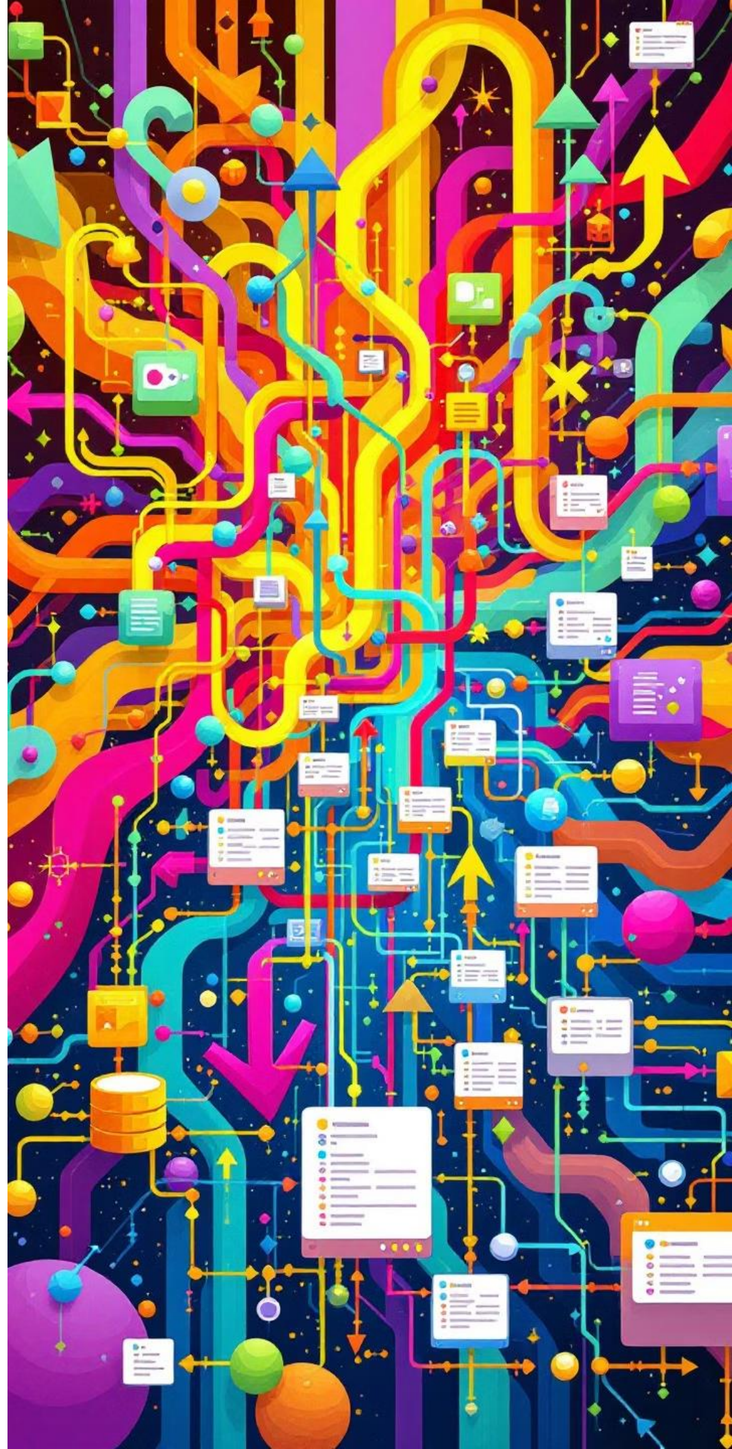




2. Desfase Objeto-Relacional

ii. Restricciones en los tipos de datos

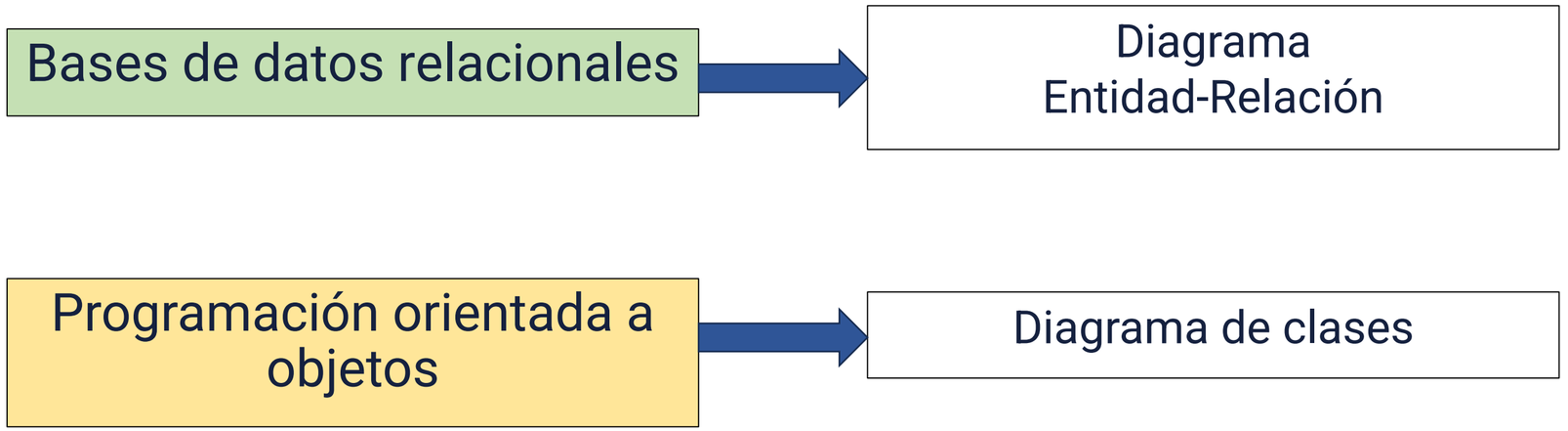
Bases de datos relacionales	Tipos de datos simples y limitados: INT, VARCHAR, DATE …
Programación orientada a objetos	Estructuras de datos complejas: colecciones de objetos (listas, arrays), referencia a otros objetos



2. Desfase Objeto-Relacional

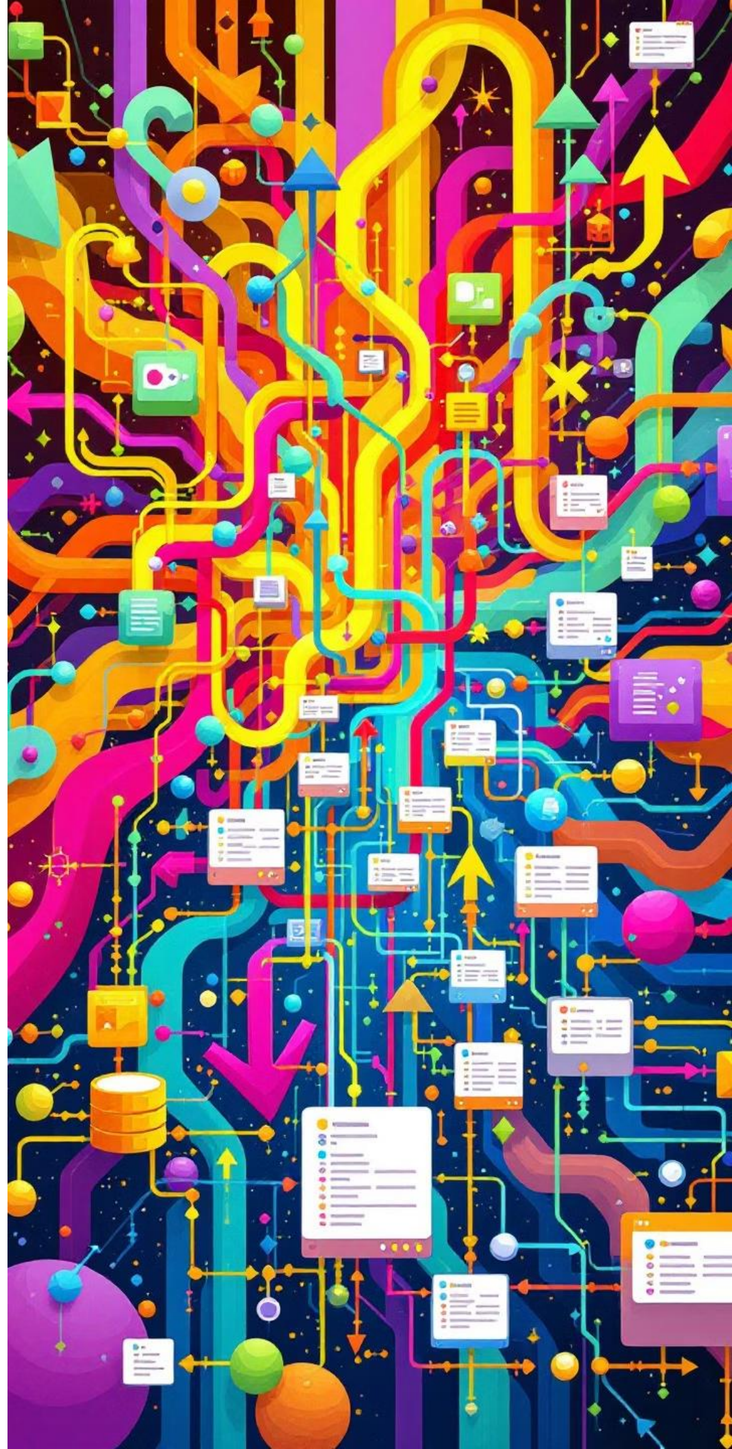
iii. Traducción entre modelos

Necesidad de diseñar y mantener dos diagramas de datos diferentes



Esta dualidad:

- Aumenta complejidad de diseño y desarrollo.
- Problemas al sincronizar cambios en ambos modelos



2. Desfase Objeto-Relacional

Soluciones al desfase objeto-relacional

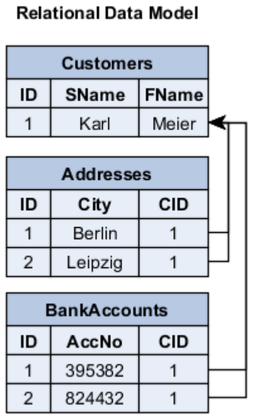
Existen dos soluciones principales para mitigar el problema del desfase objeto-relacional:

Herramientas de mapeo objeto-relacional (ORM)

(UNIDAD DE TRABAJO 3)

Establece una correspondencia entre los objetos en código y las tablas en bases de datos relacionales.

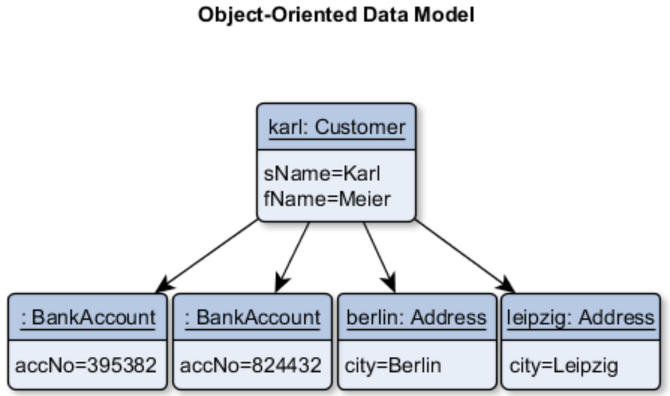
Programador trabajar con objetos en código sin preocuparse de la traducción a SQL.



Bases de datos orientadas a objetos (BDOO)

(UNIDAD DE TRABAJO 4)

Permiten almacenar objetos directamente, lo cual elimina la necesidad de un mapeo complejo entre tablas y objetos.



Preparación del entorno de trabajo

1. En esta UT accederemos desde Java a un servidor MySQL que se ejecuta en un contenedor Docker.

2. El entorno se levanta con Docker Compose y tiene dos servicios: MySQL (el gestor de bases de datos) y phpMyAdmin (administración web en http://localhost:8080).

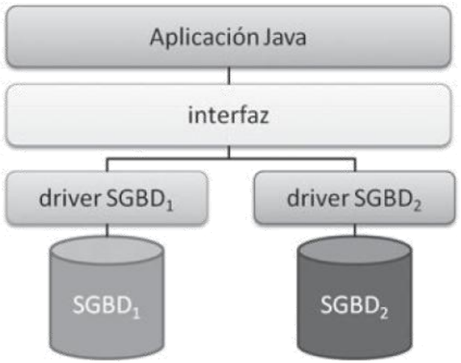
3. Para conectar desde Java hay que publicar el puerto 3306: descomenta las líneas ports del servicio db y ejecuta docker compose up -d.

4. Datos importantes de conexión: base de datos **db**, usuario **dam**, contraseña **123456**.

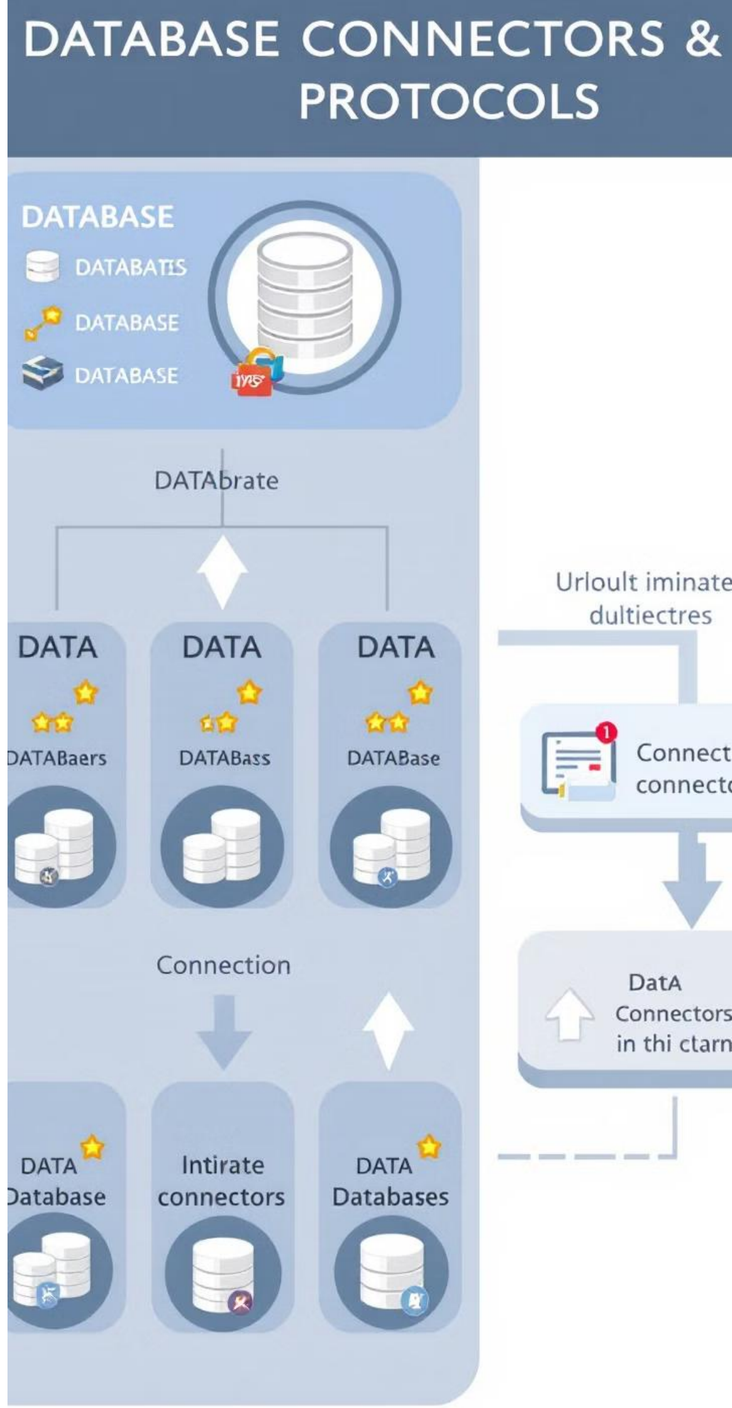
5. No olvides crear las tablas en la base de datos db (desde phpMyAdmin).

3. Protocolos de acceso a bases de datos: conectores

- Muchos servidores de bases de datos emplean **protocolos de comunicación específicos** (Sistemas Gestores de Bases de Datos - **SGBD**) que permiten la interacción con sus sistemas.
- Necesidad de aprender distintos lenguajes y métodos para trabajar con cada sistema.
- **Reducir complejidad ->** interfaces de alto nivel que ofrecen métodos estandarizados para interactuar con cualquier base de datos, independientemente de sus protocolos específicos.



Drivers permiten desarrollar arquitectura genérica donde el conector tiene interfaz común (para las aplicaciones y bases de datos), y los drivers se ocupan de las particularidades de las distintas bases de datos.





Interfaces de alto nivel

Funcionalidades principales

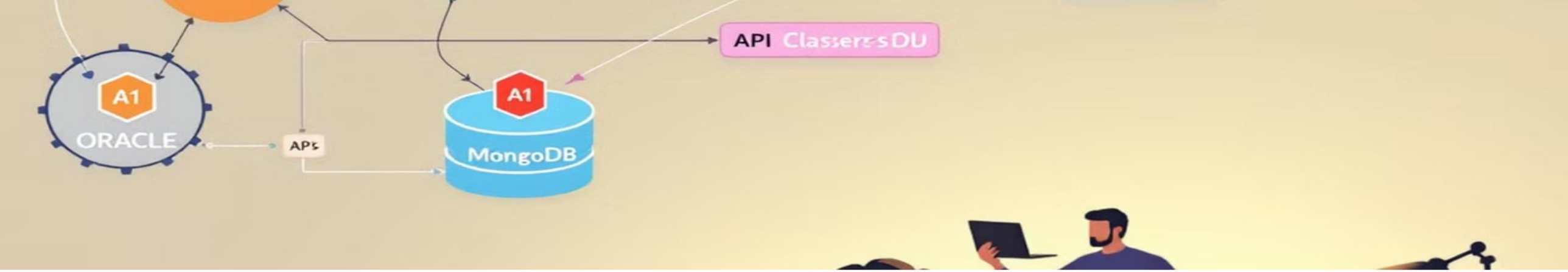
- Establecer una conexión a una base de datos de forma sencilla
- Ejecutar consultas SQL y comandos específicos sobre la base de datos
- Procesar los resultados obtenidos de las consultas, facilitando el acceso y la gestión de los datos devueltos

Ventajas para el programador

Gracias a estas interfaces, el programador puede realizar operaciones de acceso y manipulación de datos sin preocuparse de los detalles técnicos del protocolo específico de cada sistema de bases de datos.

Lenguaje de consulta estructurado basado en SQL

Todas las operaciones se expresan en SQL, el lenguaje común a los gestores relacionales. El conector se encarga de transportar cada sentencia al gestor y de devolver el resultado.



Conectores

- Al conjunto de clases encargadas de implementar la interfaz de programación de aplicaciones (API) y facilitar el acceso a una base de datos se le denomina **CONECTOR**.

- Para poder conectarse a una base de datos y lanzar consultas, una aplicación siempre necesita tener un conector asociado.

- Cuando se construye una aplicación de base de datos, **el conector oculta los detalles específicos de cada base de datos**, de modo que el programador solo debe preocuparse de los aspectos relacionados con su aplicación, olvidándose de otras consideraciones.

- La mayoría de los fabricantes ofrecen conectores para acceder a sus bases de datos.

3.1. Ejemplos más utilizados de conectores



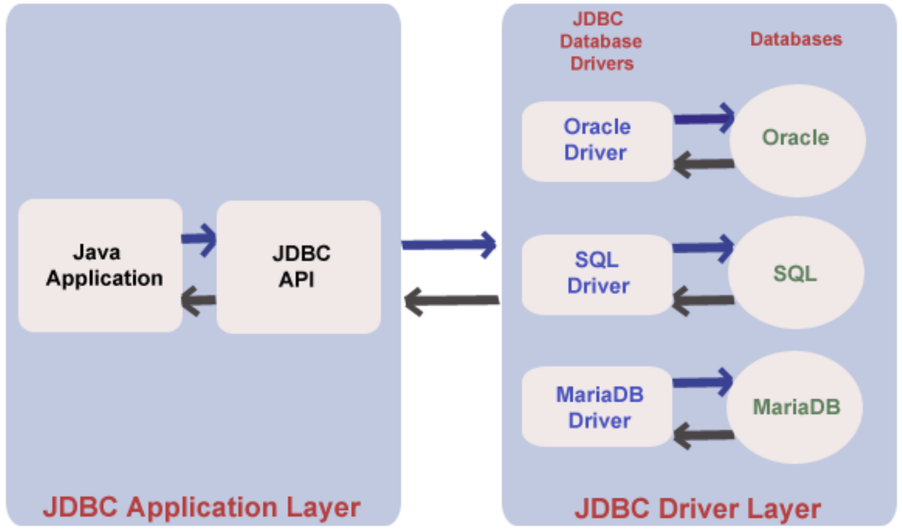
- ODBC (Open DataBase Connectivity)
- La primera arquitectura de conectores que surgió
- Desarrollada por Microsoft para Windows a principios de los noventa.
- API para el lenguaje C.
- Se usa ampliamente hoy en día tanto en Windows como en Linux y Unix.
- La mayoría de sistemas de bases de datos relacionales son compatibles con ODBC, ya que implementan drivers específicos para ello.
- En Java existió un puente JDBC-ODBC que convertía las llamadas JDBC en llamadas ODBC (se eliminó en Java 8 - 2014).

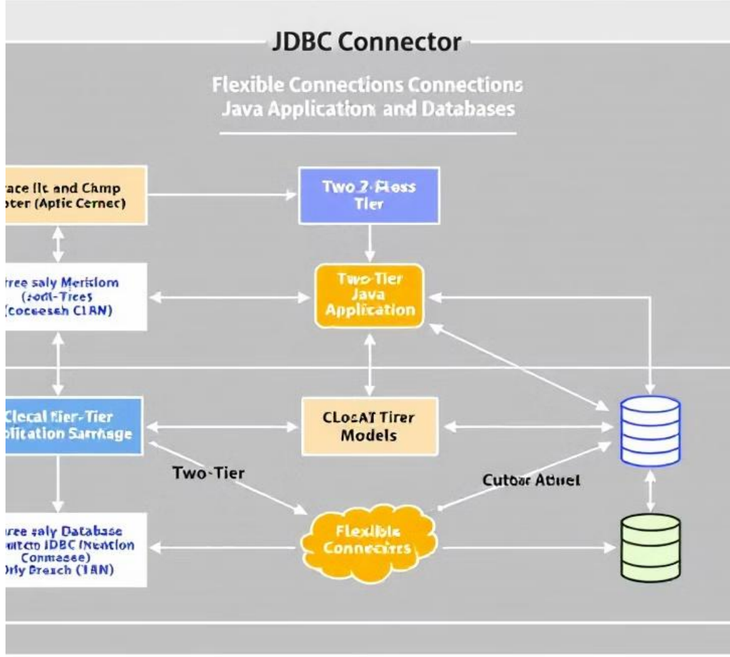


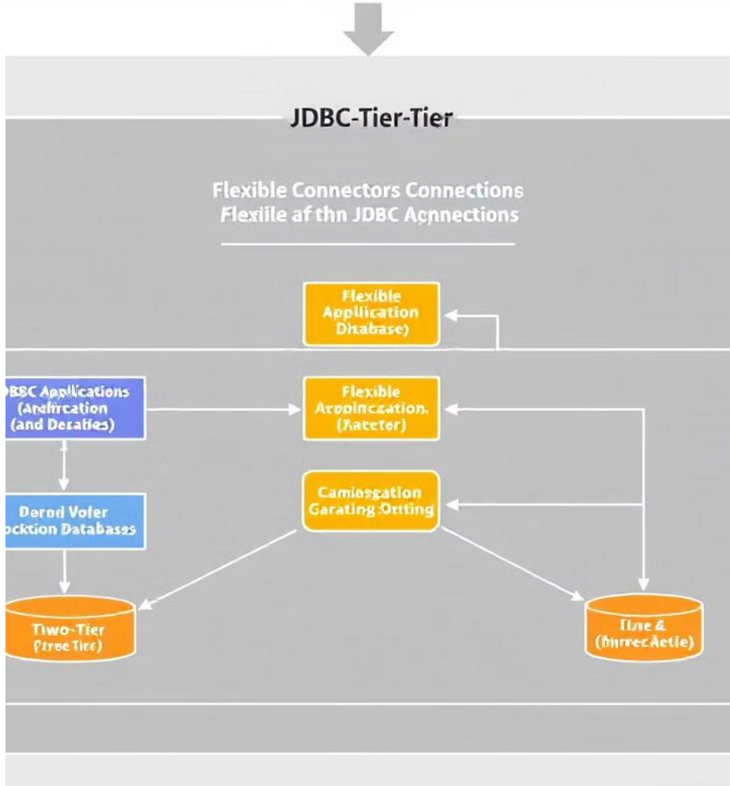
- A finales de los años noventa surgió JDBC como el equivalente a ODBC para Java.
- Interfaz desarrollada por Sun Microsystems (hoy Oracle).
- Actúa como una capa de software intermedia que permite que los programas Java se comuniquen con sistemas de gestión de bases de datos (RDBMS) que utilizan SQL.
- Independiente de plataforma y de gestor de base de datos;
- Permite que un único programa acceda a bases de datos diferentes (por ejemplo, MySQL, Oracle o PostgreSQL) sin necesidad de adaptar el código para cada sistema específico.

3.2. Conector JDBC

La arquitectura de conexión JDBC permite a las aplicaciones Java interactuar con fuentes de datos de manera flexible y escalable. Esta arquitectura puede implementarse en dos modelos principales: el modelo de dos capas y el modelo de tres capas, cada uno adaptado a distintas necesidades de configuración y estructuras de red.







Modelo de dos capas (cliente-servidor)

1

Comunicación directa

La aplicación se comunica directamente con la fuente de datos, lo que requiere un conector JDBC adecuado para gestionar la conexión con el sistema de base de datos específico.

2

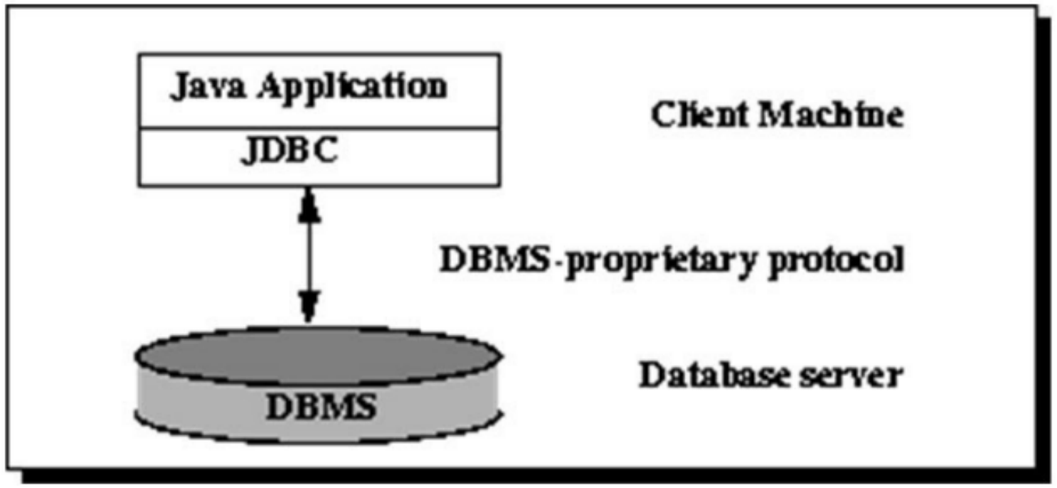
Proceso de comunicación

El usuario envía comandos o instrucciones SQL directamente a la base de datos, y la base de datos procesa esas instrucciones y devuelve los resultados al usuario.

3

Eficiencia del modelo

Este modelo es eficiente para aplicaciones de uso individual o de bajo volumen de transacciones, ya que simplifica la arquitectura al eliminar componentes intermedios.



Modelo de tres capas

1

Estructura del modelo

Añade una capa intermedia de servicios entre el cliente y la fuente de datos. Los comandos de la aplicación no se envían directamente a la base de datos. La capa intermedia gestiona y redirige los comandos a la fuente de datos.

2

Proceso de comunicación

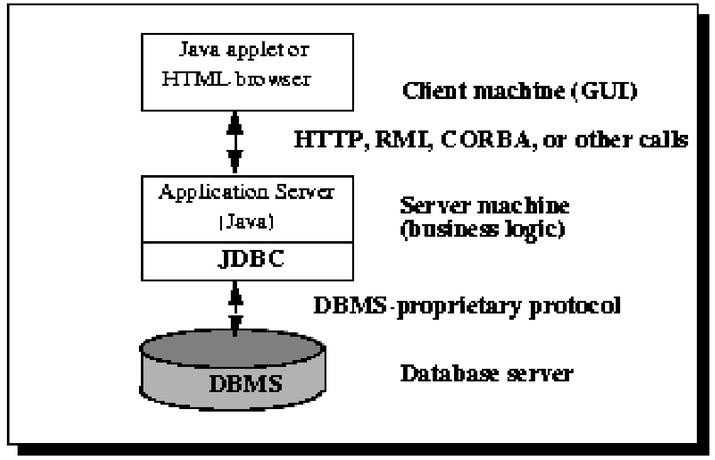
Ventajas:

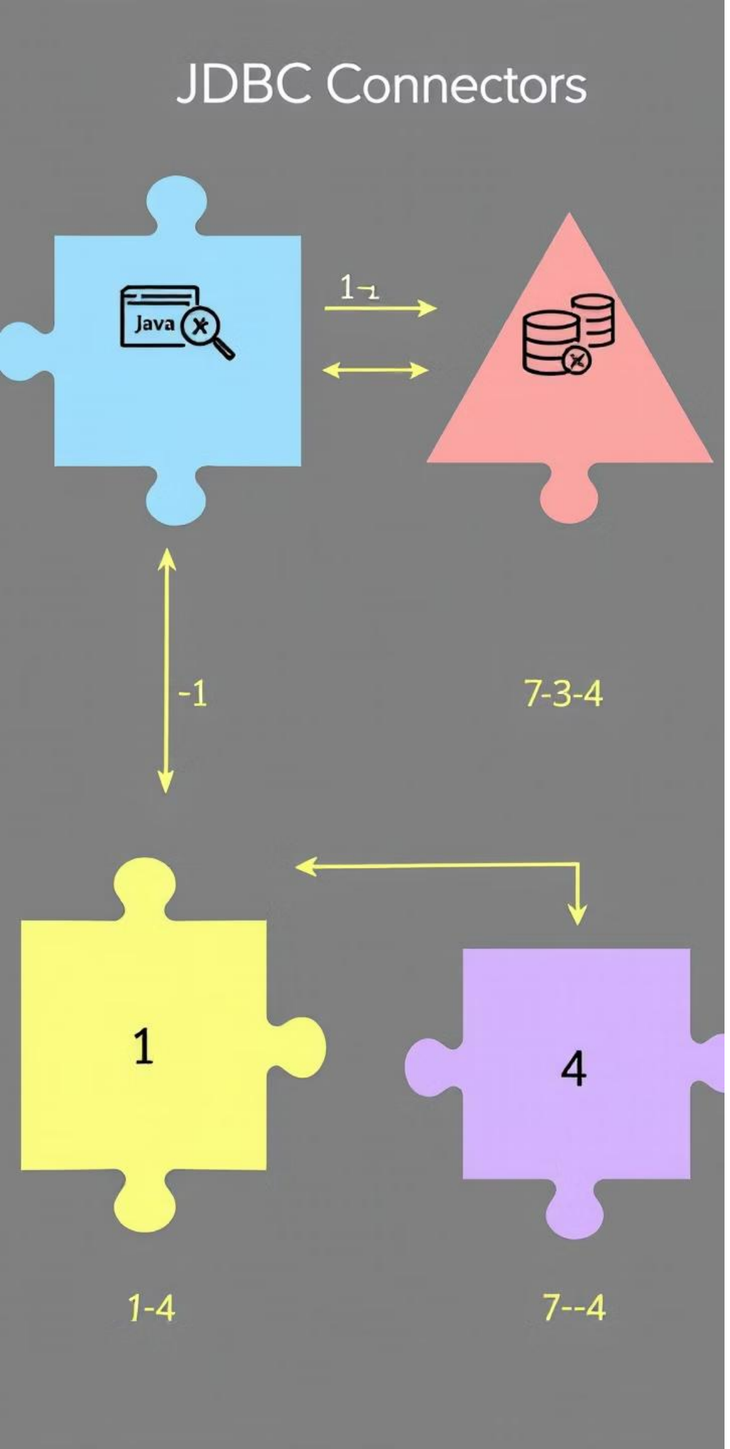
- Seguridad mejorada: Al centralizar el acceso a los datos, se pueden aplicar reglas de acceso y seguridad en la capa intermedia.
- Escalabilidad: La capa intermedia permite distribuir la carga de trabajo y gestionar un mayor volumen de transacciones.
- Separación de responsabilidades: La lógica de negocio se aísla de la lógica de presentación y de acceso a datos, facilitando el mantenimiento y la expansión de la aplicación.

3

Aplicación en entornos empresariales

Común en aplicaciones empresariales, que requieren mayor seguridad y capacidad de escalado, ya que permite controlar mejor el flujo de información entre el cliente y la fuente de datos.





Tipos de conectores JDBC

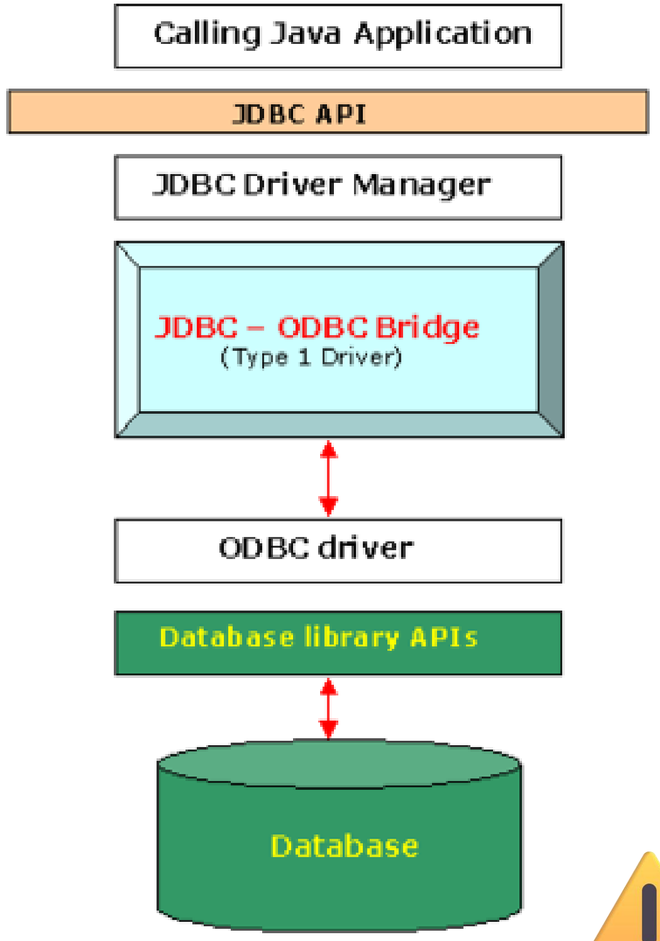
Clasificación de conectores JDBC

En el ecosistema JDBC, los conectores se clasifican en cuatro tipos, numerados del 1 al 4, según su grado de independencia de la plataforma, eficiencia y método de implementación.

Importancia de la clasificación

Esta clasificación ayuda a los desarrolladores a identificar el tipo de driver más adecuado para cada contexto de desarrollo y para cada base de datos.

Conector tipo 1: puente JDBC-ODBC

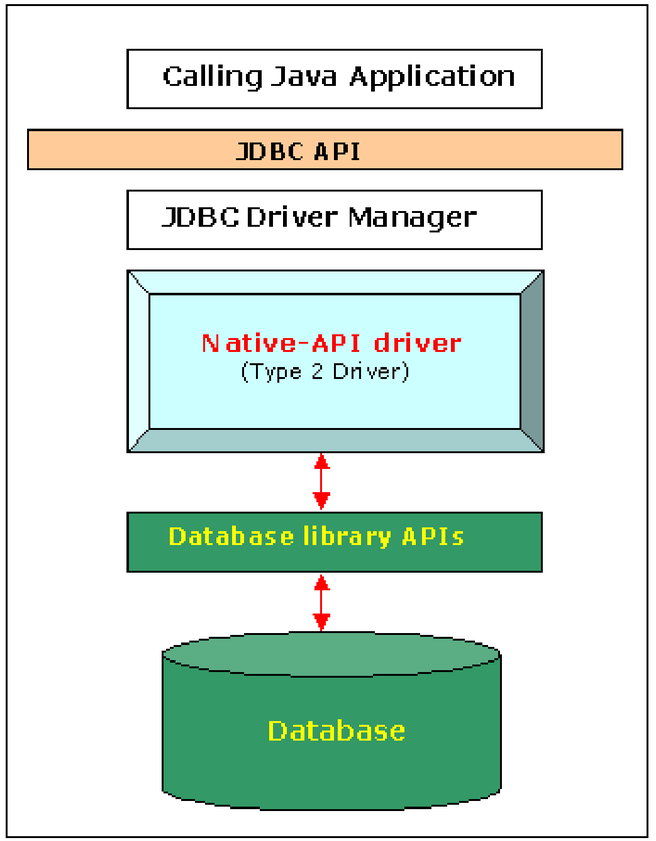


- El driver tipo 1 utiliza una API nativa estándar, que permite la comunicación entre JDBC y ODBC.
- Este tipo de driver traduce las llamadas de JDBC a invocaciones ODBC a través de librerías específicas del sistema operativo.
- Esta traducción permite a las aplicaciones Java conectarse a bases de datos que disponen únicamente de un driver ODBC.

Eliminado del JDK en Java 8 (2014). Hoy solo tiene interés histórico.

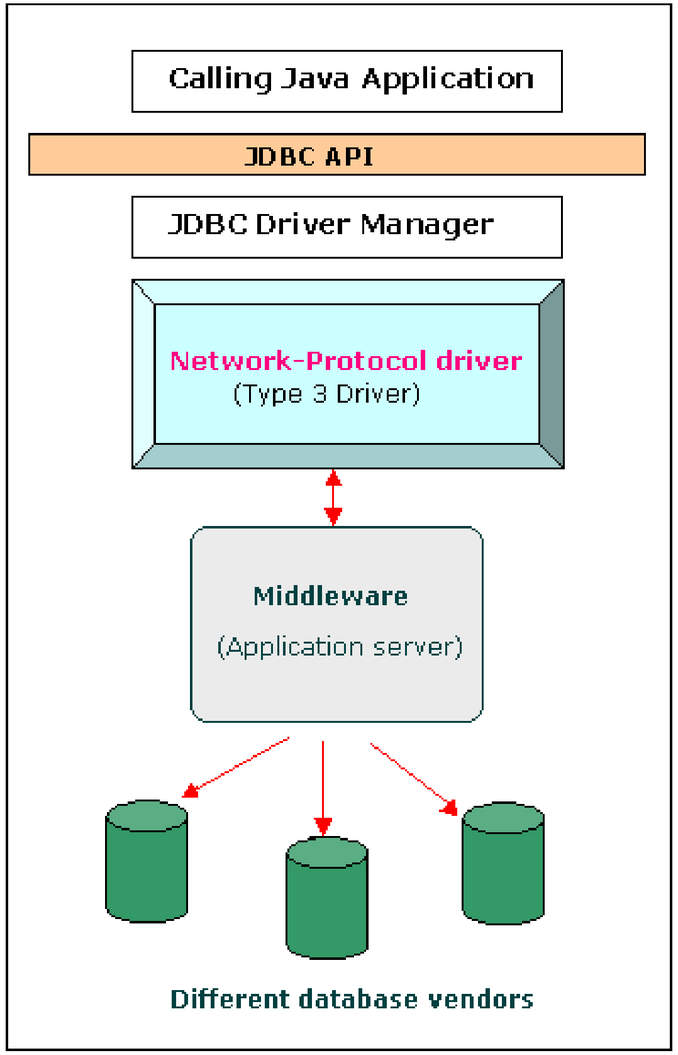
Conector tipo 2: driver parcialmente nativo

- El driver tipo 2 utiliza una API nativa de la base de datos específica y es un controlador híbrido.
- Está escrito en parte en Java y en parte en código nativo específico del sistema operativo.
- Este driver requiere de una librería cliente nativa de la base de datos con la que se desea conectar, lo que significa que no es un driver completamente Java.
- La aplicación Java realiza una llamada a la base de datos a través del driver JDBC, que luego traduce esta llamada en una invocación a la API propia del fabricante de la base de datos.



Uso residual hoy: obliga a instalar librerías nativas en cada equipo cliente.

Conector tipo 3: driver middleware (Java puro)

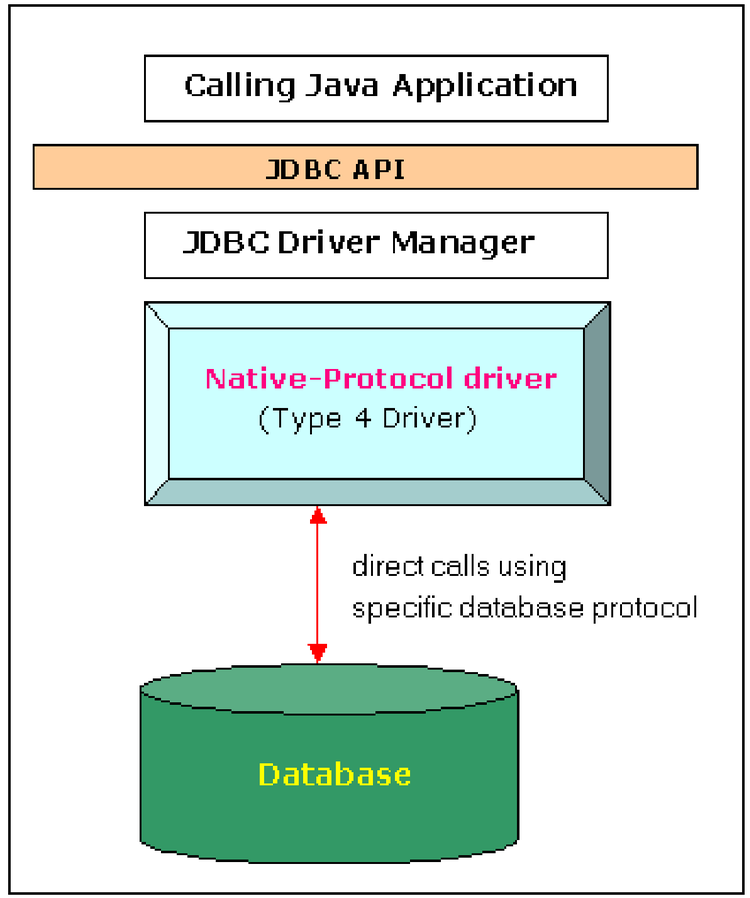


- El driver tipo 3 emplea un servidor remoto para facilitar la conexión a la base de datos.
- Este tipo de driver es un cliente Java puro que se comunica con un servidor intermedio o middleware server mediante un protocolo independiente de la base de datos, como TCP/IP.
- Este middleware server, a su vez, convierte las llamadas en un protocolo que puede interactuar directamente con la base de datos.

⚠️ **Poco habitual hoy: añade un servidor intermedio que hay que instalar y mantener.**

Conector tipo 4: driver de protocolo nativo

- El driver tipo 4 es el más eficiente y directo de los controladores JDBC.
- Este tipo de driver es proporcionado por el propio fabricante de la base de datos.
- Convierte las llamadas JDBC directamente en un protocolo de red nativo comprendido por el sistema de base de datos.
- Al ser completamente implementado en Java, este driver es independiente de la plataforma.
- Ofrece el mejor rendimiento, ya que no requiere intermediarios ni traducción adicional.



Es el tipo que usaremos: MySQL Connector/J es un driver de tipo 4.

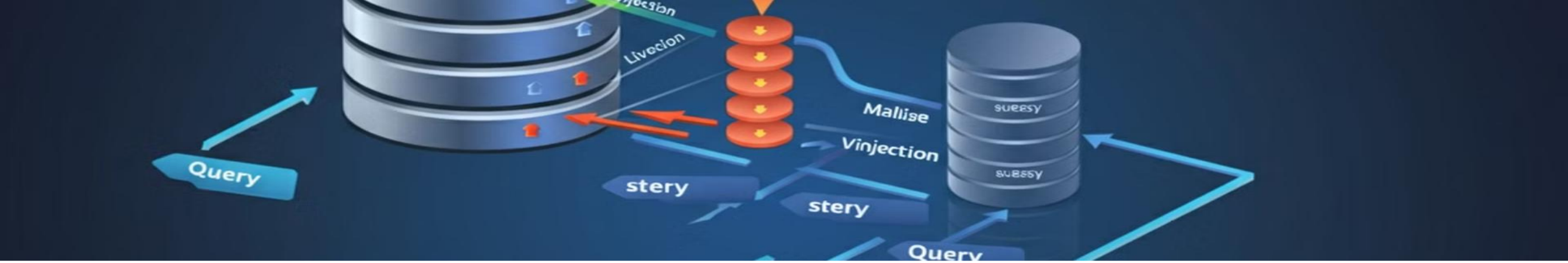
Ventajas e inconvenientes de los conectores

Ventajas

- Independencia del gestor: el mismo código sirve para MySQL, PostgreSQL u Oracle cambiando el driver y la URL.
- Estándar: la API forma parte de Java SE.
- Control total del SQL que se ejecuta.
- Es la base de los ORM (UT3).

Inconvenientes

- El SQL va dentro de cadenas de texto: los errores aparecen al ejecutar, no al compilar.
- La independencia no es total: cada gestor tiene su propio dialecto SQL.
- No resuelve el desfase objeto-relacional: hay que convertir a mano filas en objetos.
- Código repetitivo: abrir, cerrar y gestionar excepciones.



¿Cómo elegimos el conector idóneo?

1. Que corresponda al gestor: MySQL → MySQL Connector/J; MariaDB → MariaDB Connector/J; SQLite → sqlite-jdbc.
2. Preferir un driver de tipo 4 del propio fabricante.
3. Que su versión sea compatible con la del servidor y con la de Java del proyecto.

Preparación del entorno de trabajo

Añadir librerías al proyecto

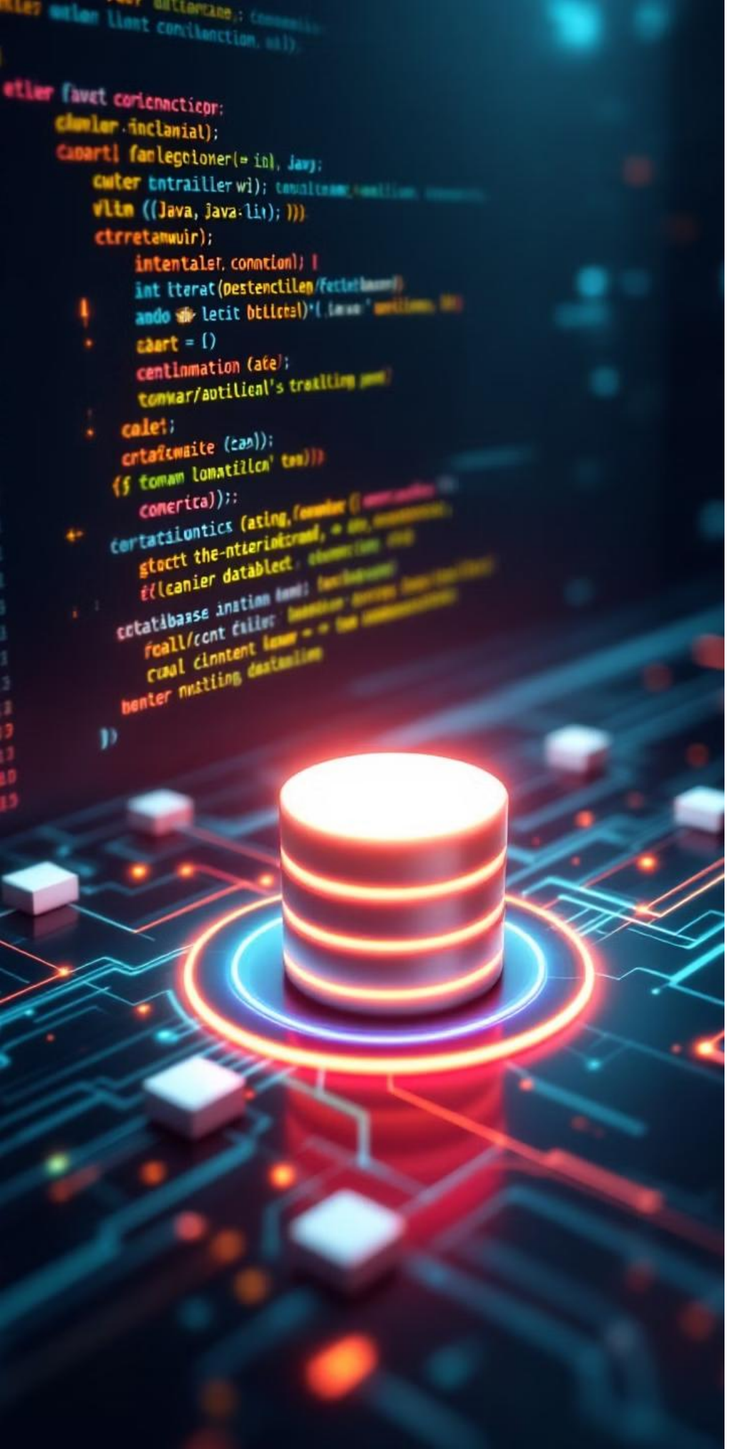
1. Obtener la API JDBC
2. Incorporar JDBC a los proyectos Java en IntelliJ.

Video: configuración de API JDBC

Crear el Proyecto como Maven y añadir dependencia

1. Configurar el Proyecto como Maven y añadir la dependencia.

Enlace a Maven

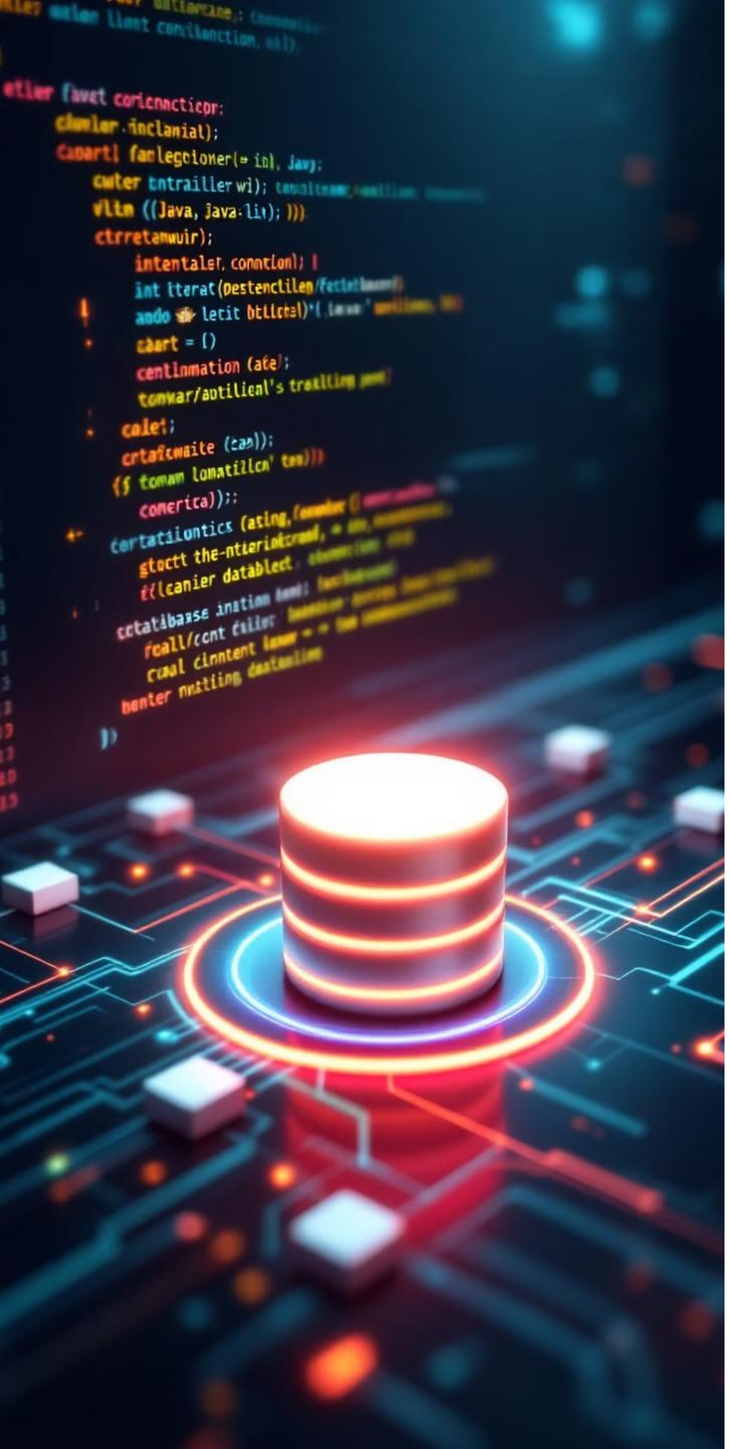


4. Establecimiento de conexiones

Primer paso fundamental para interactuar con una base de datos desde una aplicación Java.

Método getConnection()

Método de la clase DriverManager, el cual recibe como parámetro una URL específica de JDBC que identifica la base de datos con la que se desea conectar.



4. Establecimiento de conexiones

Ejemplo

```
//ImportLibraries
public class MySQLConnectionExample {
  // Datos de conexión
   private static final String URL = "jdbc:mysql://localhost:3306/nombre_base_datos";
   private static final String USER = "usuario";
   private static final String PASSWORD = "contrasena";
   public static void main(String[] args) {
      Connection connection = null;
      try {
         // Establecer conexión con la base de datos MySQL
         connection = DriverManager.getConnection(URL, USER, PASSWORD);
         if (connection != null) {
            System.out.println("Conexión a la base de datos establecida correctamente.");
         }
      } catch (SQLException e) {
         System.out.println("Error al conectar a la base de datos.");
         e.printStackTrace();
      } finally {
         // Cerrar la conexión
         if (connection != null) {
            try {
               connection.close();
               System.out.println("Conexión cerrada.");
            } catch (SQLException ex) {
               ex.printStackTrace();
            }
         }
      }
   }
}
```

Procedimiento para establecer conexiones

1 BASE DE DATOS

Preparación del entorno de trabajo

1. En esta UT accederemos desde Java a un servidor MySQL que se ejecuta en un contenedor Docker.
2. El entorno se levanta con Docker Compose y tiene dos servicios: MySQL (el gestor de bases de datos) y phpMyAdmin (administración web en http://localhost:8080).
3. Para conectar desde Java hay que publicar el puerto 3306: descomenta las líneas ports del servicio db y ejecuta docker compose up -d.
4. Datos importantes de conexión: base de datos db, usuario dam, contraseña 123456.
5. No olvides crear las tablas en la base de datos db (desde phpMyAdmin).

Docker

2 DRIVERS DE CONEXIÓN MySQL E INSTALACIÓN DEL CONECTOR

Preparación del entorno de trabajo

Añadir librerias al proyecto

1. Obtener la API JDBC
2. Incorporar JDBC a los proyectos Java en IntelliJ.

Video: configuración de API JDBC

Crear el Proyecto como Maven y añadir dependencia

1. Configurar el Proyecto como Maven y añadir la dependencia.

Enlace a Maven



Procedimiento para establecer conexiones

3

APERTURA DE CONEXIONES

Establecer una conexión utilizando la interfaz **Connection**, que actúa como un canal de comunicación entre la aplicación y la base de datos. Para obtener una conexión, se utiliza el método **getConnection()** de **DriverManager**, que tiene tres versiones (sobrecargas):

```
getConnection(String url);
getConnection(String url, Properties info);
getConnection(String url, String user, String password);
```

La URL JDBC sigue una estructura específica que incluye el protocolo (jdbc), el subprotocolo (mysql), y el sistema (host y puerto), además del nombre de la base de datos.

```
jdbc:mysql://localhost:3306/nombre_base_datos
```



Procedimiento para establecer conexiones

EJEMPLO APERTURA DE CONEXIONES

```java
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class MySQLConnectionExample {

    // Datos de conexión del contenedor Docker del aula
    private static final String URL = "jdbc:mysql://localhost:3306/db";
    private static final String USER = "dam";
    private static final String PASSWORD = "123456";

    public static void main(String[] args) {
        // try-with-resources: la conexión se cierra sola al salir del bloque
        try (Connection connection = DriverManager.getConnection(URL, USER, PASSWORD)) {
            System.out.println("Conectado a: "
                    + connection.getMetaData().getDatabaseProductName() + " "
                    + connection.getMetaData().getDatabaseProductVersion());
        } catch (SQLException e) {
            System.out.println("Error al conectar: " + e.getMessage());
        }
    }
}
```



try-with-resources cierra la conexión automáticamente, incluso si se produce un error. Es la forma recomendada desde Java 7.



Pasamos a: REALIZAR CONSULTAS



Gestores embebidos e independientes

	Independiente (servidor)	Embebido
Dónde se ejecuta	Proceso aparte (nuestro contenedor Docker)	Dentro de la propia aplicación Java
Conexión	Por red: host y puerto	Fichero local o memoria
Usuarios a la vez	Muchos	Normalmente uno
Instalación	Servidor + driver	Solo una dependencia Maven
Ejemplos	MySQL, PostgreSQL, Oracle	SQLite, H2, Apache Derby

SQLite es el gestor embebido más utilizado del mundo: está en Android, en los navegadores y en millones de aplicaciones.



Conexión a un gestor embebido: SQLite

```xml
<dependency>
    <groupId>org.xerial</groupId>
    <artifactId>sqlite-jdbc</artifactId>
    <version>3.53.2.1</version>
</dependency>
```

```java
// Crea (si no existe) el fichero agenda.db en la carpeta del proyecto
try (Connection con = DriverManager.getConnection("jdbc:sqlite:agenda.db")) {
        System.out.println("Conectado a: " + con.getMetaData().getDatabaseProductName());
}
```

Mismo código JDBC que con MySQL: cambian el driver y la URL. SQLite no usa usuario ni contraseña.

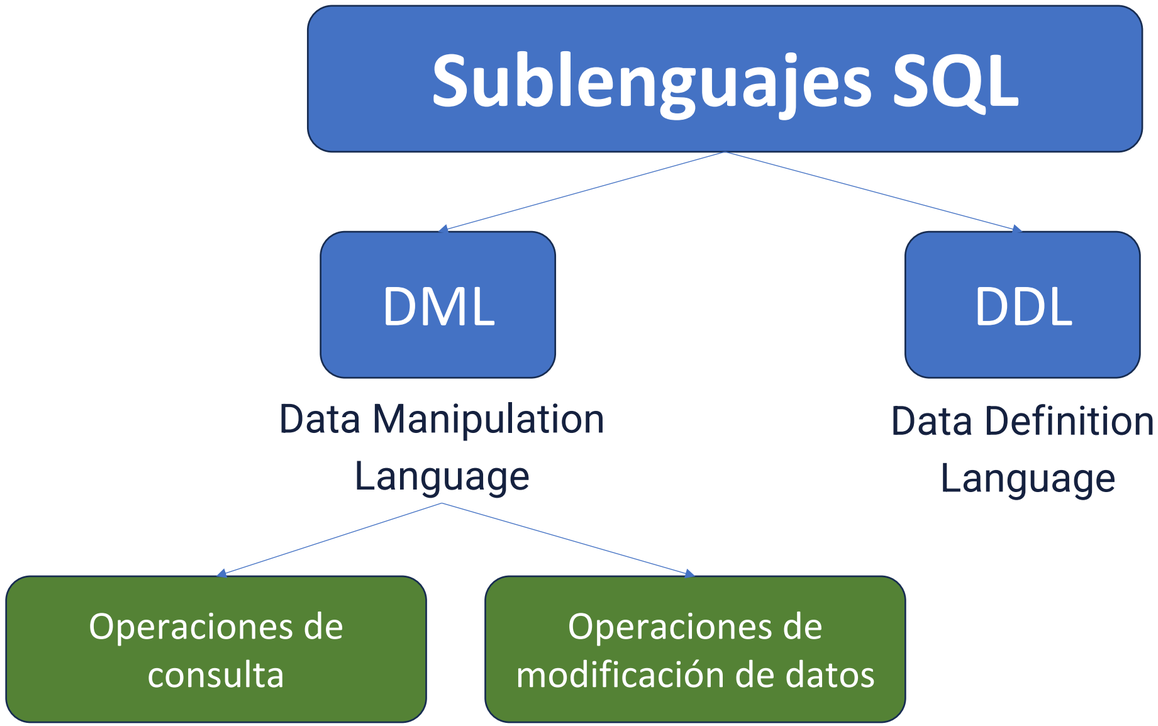


Desarrolla el conector en JAVA (utilizando la API JDBC) a la base de datos MySQL que has creado anteriormente

TAREA 01

5. Operaciones básicas

- Principales tipos de sentencias SQL con JDBC.
- En SQL hay varios sublenguajes, y a cada uno de ellos pertenecen varios tipos de sentencias.





Resumen para ejecutar cualquier sentencia SQL con JDBC

`Class.forName(nombre del driver); // No necesario desde JDBC 4.0 (Java SE 6)`		Cargar *driver*
`Connection c = DriverManager.getConnection(datos de conexión);`		Crear conexión
`Statement s = c.createStatement();`		Crear sentencia

`ResultSet rs = s.executeQuery(consulta);`
`while(rs.next()) {`
`  ...`
`}`
`rs.close();`
	`int res = s.executeUpdate (sent. DML);`
	(o bien)
	`boolean res = s.execute (sent. DDL);`
		Ejecutar sentencia
		Si consulta, obtener resultados fila a fila y cerrar lista de resultados

`s.close();`		Cerrar sentencia
`c.close();`		Cerrar conexión

Consulta

Sentencias SELECT se ejecutan con executeQuery()

Modificación

Sentencias UPDATE, DELETE, INSERT se ejecutan con executeUpdate()

Definición

Sentencias DDL se ejecutan con execute()

5.1. La interfaz Statement

Antes de pasar a describir los diferentes tipos de sentencias que se pueden ejecutar sobre base de datos relacionales con JDBC, hemos de conocer el significado de la **interfaz Statement**

- **Herramienta fundamental** en JDBC para ejecutar sentencias SQL.
- Permite ejecutar **consultas SELECT**, **sentencias DML (**INSERT, UPDATE y DELETE**)** y **sentencias DDL (**CREATE, ALTER**)**.
- Se crea mediante el método **createStatement()** de la clase Connection.





5.1. La interfaz Statement

METODO	FUNCIONALIDAD
ResultSet executeQuery(String sql)	Ejecuta una consulta SELECT de SQL y devuelve un objeto ResultSet que permite acceder a los resultados.
ResultSet getResultSet()	Obtiene el conjunto de resultados de una consulta SELECT y de otros tipos de sentencias que se verán más adelante, como procedimientos almacenados.
int executeUpdate(String sql)	Ejecuta una sentencia SQL que modifica los contenidos de la base de datos (como INSERT, UPDATE o DELETE). Devuelve un valor int que indica la cantidad de filas afectadas en la base de datos.
boolean execute(String sql)	Permite ejecutar cualquier tipo de sentencia SQL, incluidas las sentencias de definición de datos (DDL) como CREATE, ALTER o DROP. Es el método que hay que utilizar preferentemente para este tipo de sentencias. Si se trata de una sentencia DDL, devuelve false, ya que no produce un conjunto de resultados. Para sentencias DDL, también se podría utilizar el método executeUpdate, y devolvería cero. Si se ejecuta una consulta, devuelve true y los resultados pueden recuperarse mediante getResultSet().



5.1. La interfaz Statement

Ejemplo (con errores)

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.Statement;
import java.sql.SQLException;

public class JDBC_create  table {
    public static void main(String[] args){
    (...) // Se omite declaración de variables para los datos de conexión
     try(
      Connection c = DriverManager.getConnection(urlconnection,user,pwd);
      Statement s = c.createStatement()) {
      s.execute("CREATE TABLE CLIENTES (DNI CHAR(9)NOT NULL, APELLIDOS VARCHAR(32) NOT
      NULL, CP CHAR(5),PRIMARYKEY(DNI))");
      //...
     } catch (SQLException e){
         muestraErrorSQL(e);
     } catch(Exception e){
         e.printStackTrace(System.err);
     }
         }
}
```

Sublenguajes SQL

DML

Data Manipulation Language

Operaciones de consulta

Operaciones de modificación de datos

DDL

Data Definition Language

execute ()



5.2. Ejecución de sentencias de descripción de datos (DDL)

- El lenguaje de definición de datos (DDL) permite gestionar la estructura de la base de datos mediante sentencias SQL para crear, modificar y eliminar objetos como tablas, vistas, índices y otros elementos en una base de datos relacional.
- Las sentencias DDL, como **CREATE, ALTER y DROP**, se ejecutan con el método **execute().**

execute ()

5.2. Ejecución de sentencias de descripción de datos (DDL)

Ejemplo

```java
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.Statement;
import java.sql.SQLException;

public class JDBC_CreateTable {
    public static void main(String[] args) {
        // Se omiten las variables explícitas de conexión

        try (
            Connection c = DriverManager.getConnection(urlConnection, user, pwd);
            Statement s = c.createStatement()
        ) {
            // Sentencia SQL para crear la tabla CLIENTES
            s.execute("CREATE TABLE CLIENTES (" +
                      "DNI CHAR(9) NOT NULL, " +
                      "APELLIDOS VARCHAR(32) NOT NULL, " +
                      "CP CHAR(5), " +
                      "PRIMARY KEY (DNI))");
            System.out.println("Tabla CLIENTES creada exitosamente.");

        } catch (SQLException e) {
            muestraErrorSQL(e);
        } catch (Exception e) {
            e.printStackTrace(System.err);
        }
    }

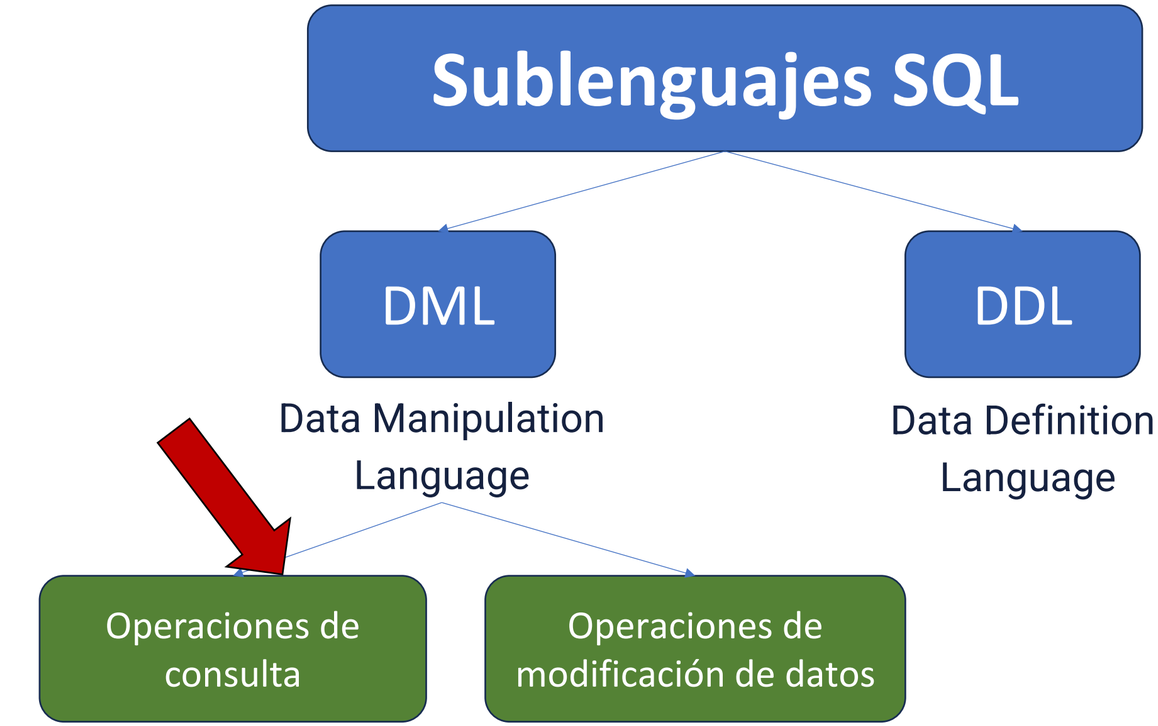
    private static void muestraErrorSQL(SQLException e) {
        System.err.println("SQL Exception: " + e.getMessage());
        System.err.println("SQL State: " + e.getSQLState());
        System.err.println("Vendor Error Code: " + e.getErrorCode());
    }
}
```

Crea una tabla (“clientes”) en tu base de datos original

- Utiliza la interfaz “Statement” y el método execute.
- Recuerda que ya debes tener creadas las siguientes tablas:
  - Agentes
  - Llamadas emitidas
  - Tarjetas telefonicas

```
s.execute("CREATE TABLE clientes (" +
          "DNI CHAR(9) NOT NULL, " +
          "APELLIDOS VARCHAR(32) NOT NULL, " +
          "CP CHAR(5), " +
          "PRIMARY KEY (DNI))");
```

TAREA 02



executeQuery ()



5.3. Ejecución de consultas (DML - consultas)

executeQuery()

En JDBC, las consultas se refieren específicamente a las sentencias SQL de tipo SELECT. Estas consultas permiten recuperar datos almacenados en una base de datos y se ejecutan utilizando el método **executeQuery()** de la interfaz Statement. Este método devuelve un objeto **ResultSet**, que contiene los resultados de la consulta como un conjunto de filas organizadas en columnas.

1

executeQuery()

Ejecuta consultas SQL SELECT

2

Devuelve ResultSet

Contiene los resultados de la consulta

3

Filas y columnas

Los resultados se organizan en filas y columnas

5.3. Ejecución de consultas (DML - consultas)

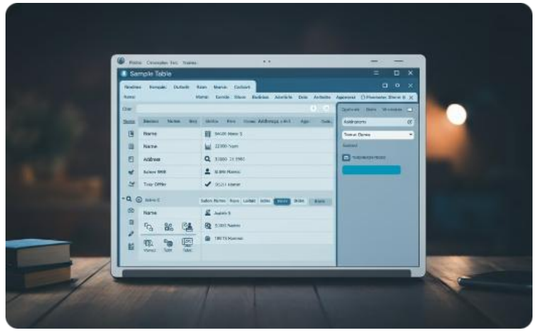
ResultSet y el cursor de navegación

Un ResultSet representa los resultados de una consulta SQL y contiene un cursor



Cursor de navegación

El cursor se posiciona en la fila actual dentro del conjunto de resultados.



Acceso a datos

Se puede acceder a los datos de cada fila usando métodos específicos.
Se avanza por filas sucesivas con el método next()

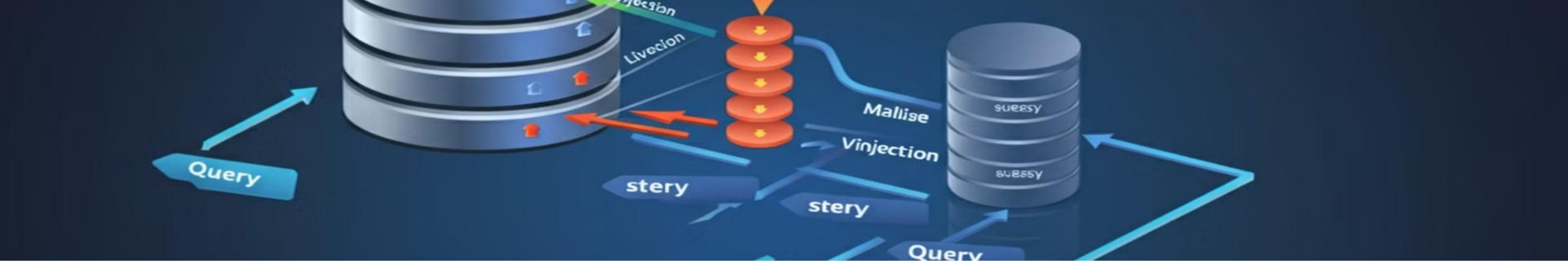
next()



Cierre del ResultSet

Es importante cerrar el ResultSet cuando ya no se necesite.

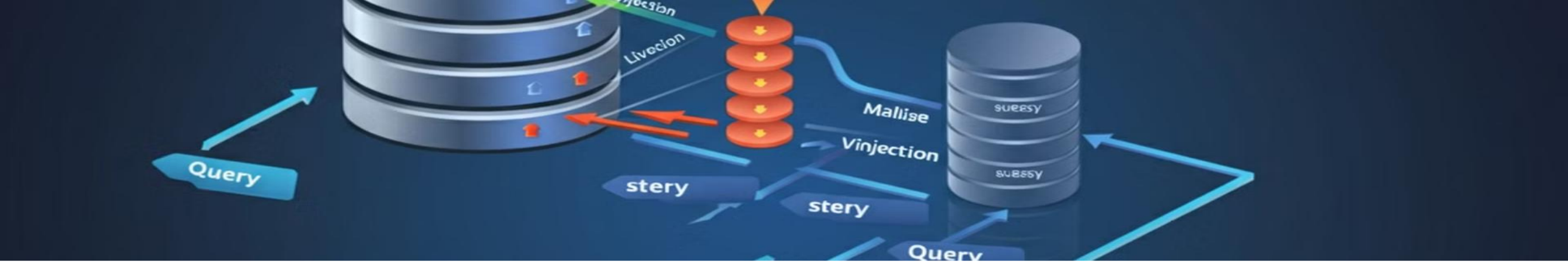
close()



5.3. Ejecución de consultas (DML - consultas)

ResultSet y cursor de navegación (métodos)

METODO	FUNCIONALIDAD
boolean next()	El cursor del ResultSet puede apuntar a cualquier fila suya. Puede, además, apuntar también a una posición especial justo antes de la primera fila y a otra posición especial justo después de la última fila. El cursor apunta inicialmente a la posición especial de antes de la primera fila. Este método mueve el cursor a la siguiente posición y devuelve true, a menos que el cursor esté en la última fila o en la posición especial tras ella, en cuyo caso devuelve false.
getXXX(int) **getXXX(String)**	Obtiene el contenido de la columna especificada de la fila actual. Se puede especificar una columna por su posición o por su nombre. Hay distintas funciones para distintos tipos: getInt(), getString(), getDate(), etc
close()	Cierra el ResultSet. Debería hacerse siempre.



Ejemplo básico de ejecución de consulta SELECT

```
Statement statement = connection.createStatement();
ResultSet resultSet = statement.executeQuery("SELECT nombre, edad FROM usuarios");

while (resultSet.next()) {
    String nombre = resultSet.getString("nombre");
    int edad = resultSet.getInt("edad");
    System.out.println("Nombre: " + nombre + ", Edad: " + edad);
}
```

Realiza una consulta a la base de datos existente

- **Consulta la tabla “AGENTES” para que saque por consola todo el contenido de la tabla de manera similar a la siguiente captura de pantalla.**
- **Utiliza executeQuery() para la consulta, y ResultSet para los resultados**

```
Código Agente: 1, Nombre Agente: Jose Luis Torrente, Frase Clave: El Cuerpo ya no tiene la suerte de tenerme entre sus filas
Código Agente: 2, Nombre Agente: Jason Bourne, Frase Clave: Me acuerdo de todo
Código Agente: 3, Nombre Agente: James Bond, Frase Clave: Excelente. Da para una copa
Código Agente: 4, Nombre Agente: Harry Tasker, Frase Clave: Mi caballo se está cansando
Código Agente: 5, Nombre Agente: John MccLane, Frase Clave: NNNNNNNNK!!!. Lo siento Hans, no es correcto
Código Agente: 6, Nombre Agente: Casey Ryback, Frase Clave: También sé cocinar
Código Agente: 7, Nombre Agente: Ethan Hunt, Frase Clave: No desde que una bomba voló el Kremlin
Código Agente: 8, Nombre Agente: John Rambo, Frase Clave: Están todos muertos, señor
Código Agente: 9, Nombre Agente: Bryan Mills, Frase Clave: Lo que sí tengo es una serie de habilidades concretas
```

TAREA 03

5.3. Ejecución de consultas (DML - consultas)

Sentencias preparadas

- Durante lo visto anteriormente, las sentencias SQL utilizadas eran fijas y se construían como cadenas constantes. Sin embargo, en aplicaciones reales, **es común que las sentencias SQL incluyan variables**.

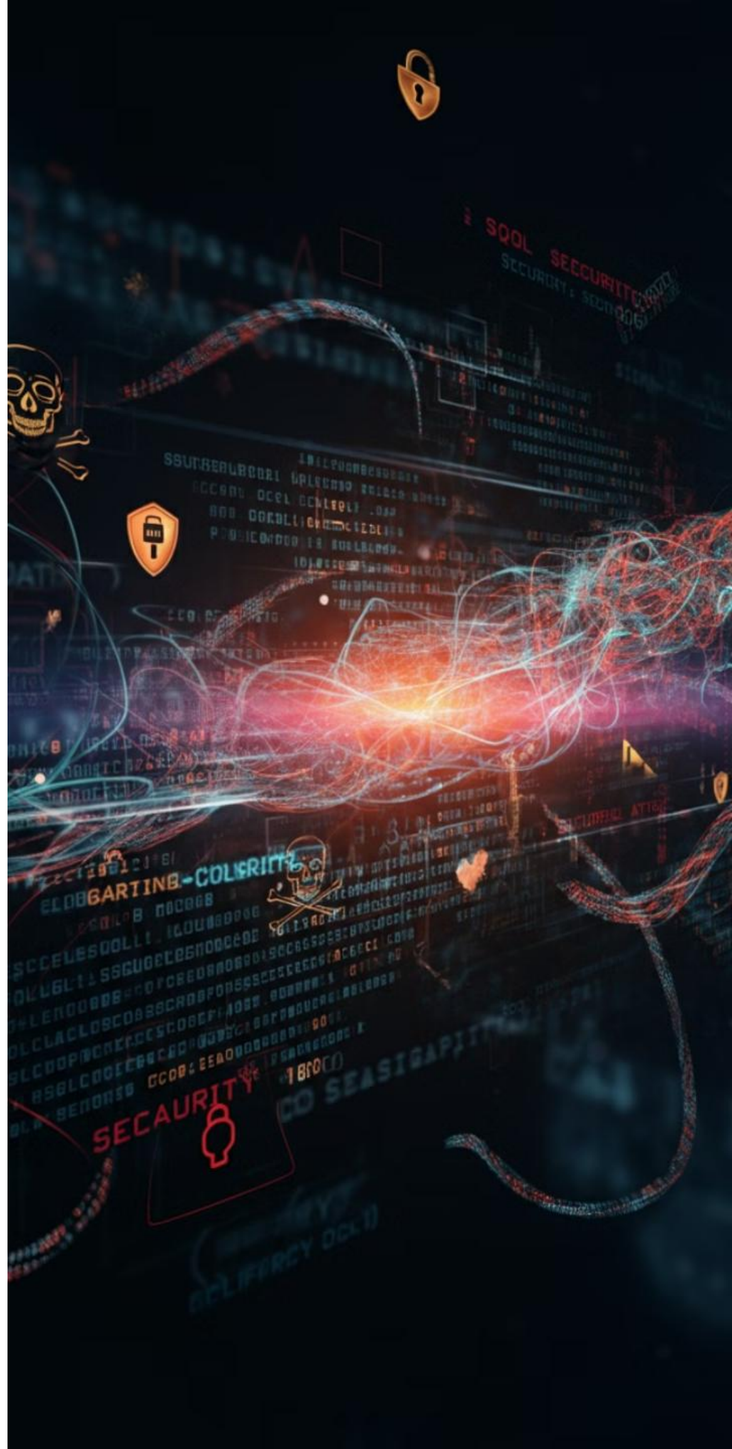
- Una forma de incluir variables sería construyendo la sentencia en un String y concatenando las variables necesarias.

```
String consulta = "SELECT * FROM CLIENTES WHERE DNI='" + dni + "'";
```

PROBLEMAS DE SEGURIDAD Y DE RENDIMIENTO

- **Seguridad**: una sentencia SQL construida en tiempo de ejecución mediante variables es vulnerable a ataques de inyección de SQL.

- **Rendimiento**: cada vez que una consulta se envía al sistema gestor de base de datos (SGBD), esta debe ser compilada para crear un plan de ejecución



5.3. Ejecución de consultas (DML - consultas)

Ventajas de sentencias preparadas

PreparedStatement



Envío único

La sentencia se envía al servidor una sola vez y se compila.



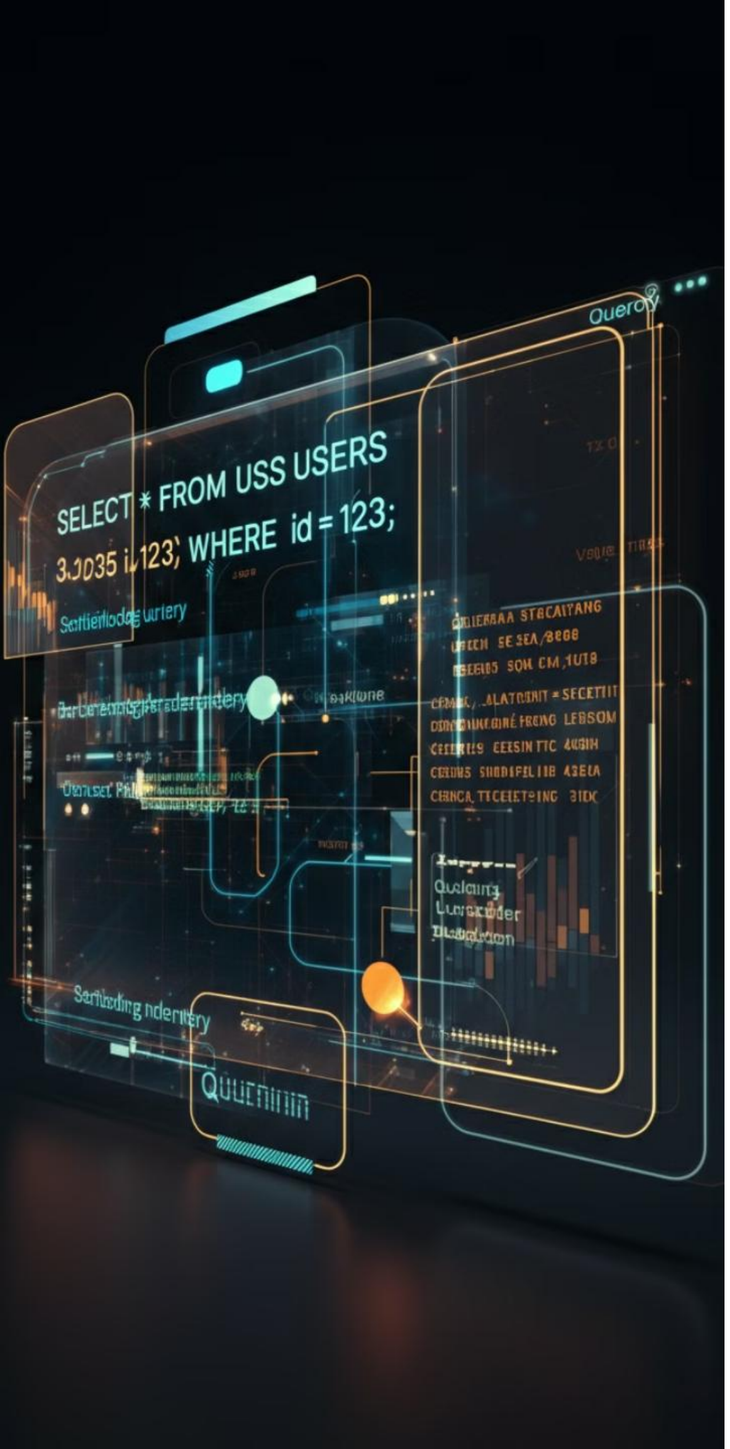
Marcadores de parámetros (placeholders)

Usa marcadores en lugar de construir la sentencia dinámicamente.
Se colocan los marcadores donde se necesitan variables



Reutilización de plan

El plan de ejecución se reutiliza en cada ejecución de la consulta.



5.3. Ejecución de consultas (DML - consultas)

Uso de PreparedStatement

1

Definir consulta SQL

Usar ? como marcadores de posición para valores de variables.

Ejemplo: String sql = “SELECT * FROM CLIENTES WHERE DNI = ?”;

2

Obtener PreparedStatement

Usar método preparedStatement() de la clase Connection. En este momento, la orden queda “impresa” y ya nadie puede cambiarla.

Ejemplo: PreparedStatement ps = connection.prepareStatement(sql);

3

Reemplazar marcadores

Asignar valores correspondientes a cada ? al ejecutar la consulta. Es decir, rellenar las casillas.

Ejemplo: ps.setString(1, dni);

5.3. Ejecución de consultas (DML - consultas)

Métodos clave de PreparedStatement

METODO	FUNCIONALIDAD
ResultSet executeQuery() **int executeUpdate()** **boolean execute()**	Estos 3 métodos no tienen como parámetro la consulta, esta se le pasó al constructor
setXXX(int pos, YYY valor)	Se utilizan para asignar un valor a un placeholder determinado, dado por su posición, siendo 1 la posición del primero. Los hay con distintos nombres, dependiendo del tipo.
setNull(int pos, int tipoSQL)	Asigna valor NULL a una columna. Debe indicarse el tipo. Los posibles tipos están definidos en la clase java.sql.Types.

Ejemplo de uso PreparedStatement

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.SQLException;

public class EjemploSentenciaPreparada {
    // Se omite la declaración explícita de variables para los datos de conexión
    public static void main(String[] args) {
        Connection connection = null;
        PreparedStatement preparedStatement = null;

        try {
            // Establecer la conexión
            connection = DriverManager.getConnection(URL, USER, PASSWORD);

            // Definir la sentencia SQL con marcadores de posición
            String sql = "INSERT INTO CLIENTES1 (DNI, APELLIDOS, CP) VALUES (?, ?, ?)";
            preparedStatement = connection.prepareStatement(sql);

            // Insertar el primer cliente
            int i = 1;
            preparedStatement.setString(i++, "123456789");
            preparedStatement.setString(i++, "Garcia");
            preparedStatement.setInt(i++, 28001);
            preparedStatement.executeUpdate();

            // Insertar el segundo cliente
            i = 1;
            preparedStatement.setString(i++, "987654321");
            preparedStatement.setString(i++, "Lopez");
            preparedStatement.setInt(i++, 28002);
            preparedStatement.executeUpdate();

            System.out.println("Datos insertados correctamente.");

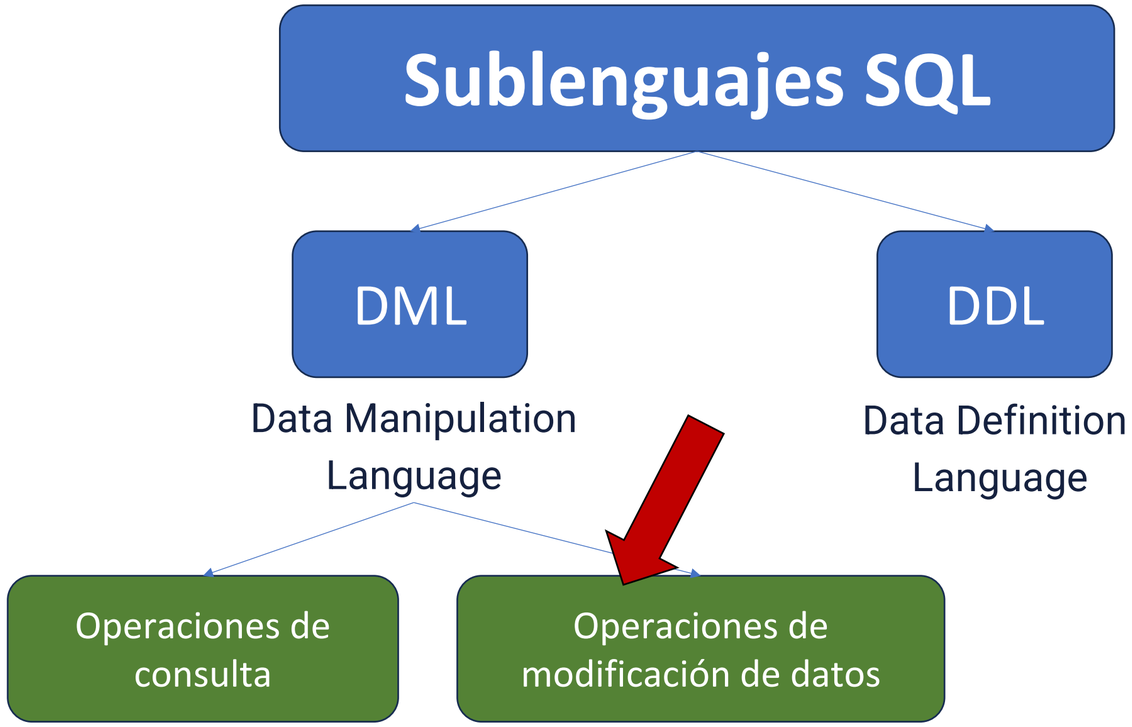
        } catch (SQLException e) {
            e.printStackTrace();
        } finally {
            // Cerrar el PreparedStatement y la conexión
            try {
                if (preparedStatement != null) preparedStatement.close();
                if (connection != null) connection.close();
            } catch (SQLException ex) {
                ex.printStackTrace();
            }
        }
    }
}
```

Realiza una consulta a la base de datos existente

- Consulta la tabla “agentes” para que busque el agente con CODIGO AGENTE = 7.
- Utiliza sentencias preparadas (preparedStatement) y executeQuery() para realizar la consulta. Gestiona el resultado con ResultSet.

```
Código Agente: 7, Nombre Agente: Ethan Hunt, Frase Clave: No desde que una bomba voló el Kremlin
```

TAREA 04



executeUpdate ()



5.4. Ejecución de sentencias DML de modificación de datos

Operaciones de modificación

INSERT, UPDATE y DELETE alteran el contenido de la base de datos.

Método executeUpdate()

Devuelve número de filas afectadas, verificando éxito de modificación.

Seguimiento de cambios

Permite saber cuántos registros fueron modificados en la base de datos.



5.4. Ejecución de sentencias DML de modificación de datos

Ejemplo

```java
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.Statement;
import java.sql.SQLException;

public class JDBCInsert {
    public static void main(String[] args) {
        // Se omite la declaración explícita de variables para los datos de conexión

        try (
            Connection c = DriverManager.getConnection(urlConnection, user, pwd);
            Statement s = c.createStatement()
        ) {
            // Ejecución de sentencias INSERT con múltiples valores
            int nFil = s.executeUpdate(
                "INSERT INTO CLIENTES (DNI, APELLIDOS, CP) VALUES " +
                "('78901234X', 'NADALES', '44126'), " +
                "('89012345E', 'HOJAS', null), " +
                "('56789012B', 'SAMPER', '29730'), " +
                "('09876543K', 'LAMIQUIZ', null);"
            );
            System.out.println(nFil + " Filas insertadas.");

        } catch (SQLException e) {
            muestraErrorSQL(e);
            System.err.println("Código de error SQL específico: " + e.getErrorCode());
        } catch (Exception e) {
            e.printStackTrace(System.err);
        }
    }
}
```



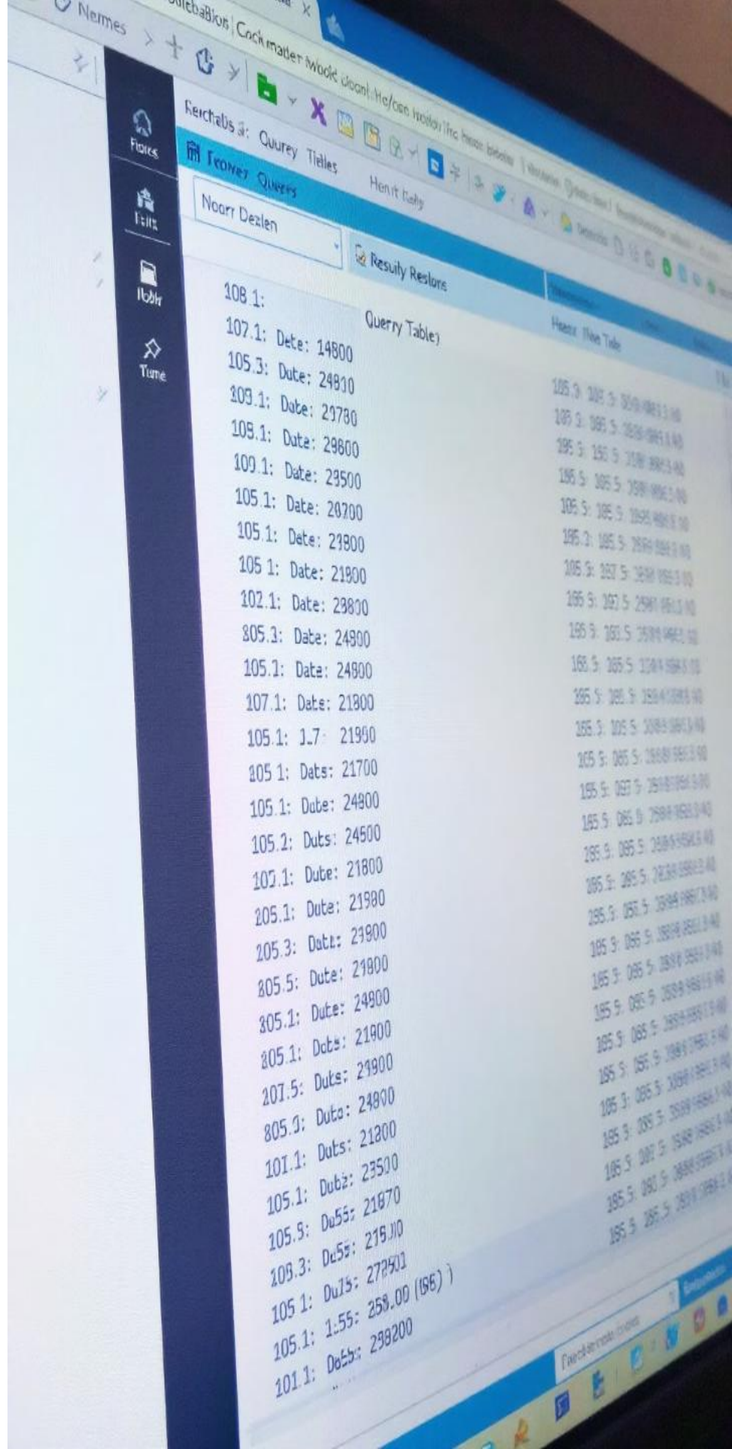
Realiza una modificación de datos sobre una base de datos existente

- **Ejecutar una sentencia INSERT para agregar múltiples filas en la tabla CLIENTES.**
- **En lugar de la consola, usarás Java Swing. En este caso, podemos mostrar el resultado en un JOptionPane (un cuadro de diálogo)..**

TAREA 05

6. Utilización del resultado de una consulta

- En JDBC, el resultado de una consulta SQL se almacena en un objeto **ResultSet.**
- ResultSet representa una tabla de datos con las filas y columnas obtenidas de la base de datos.
- Este objeto permite acceder a cada fila y columna mediante métodos de recuperación de datos y puede iterarse utilizando un bucle **while** que avanza el cursor con **next()**.





6.1. Tipos de ResultSet y su configuración

Configuración por defecto

Los ResultSet en JDBC no permiten actualizar la base de datos y solo pueden moverse en una dirección (hacia adelante). -> Equivale a *TYPE_FORWARD_ONLY + CONCUR_READ_ONLY*

ResultSet avanzados

Es posible crear ResultSet que admitan desplazamiento bidireccional y permitan realizar modificaciones en la base de datos.

Parámetros adicionales

Al crear el Statement o PreparedStatement, se utilizan dos parámetros adicionales para configurar el ResultSet: el tipo de desplazamiento y la concurrencia.

```
con.createStatement(tipo, concurrencia)
con.prepareStatement(sql, tipo, concurrencia)
```

6.1. Tipos de ResultSet y su configuración

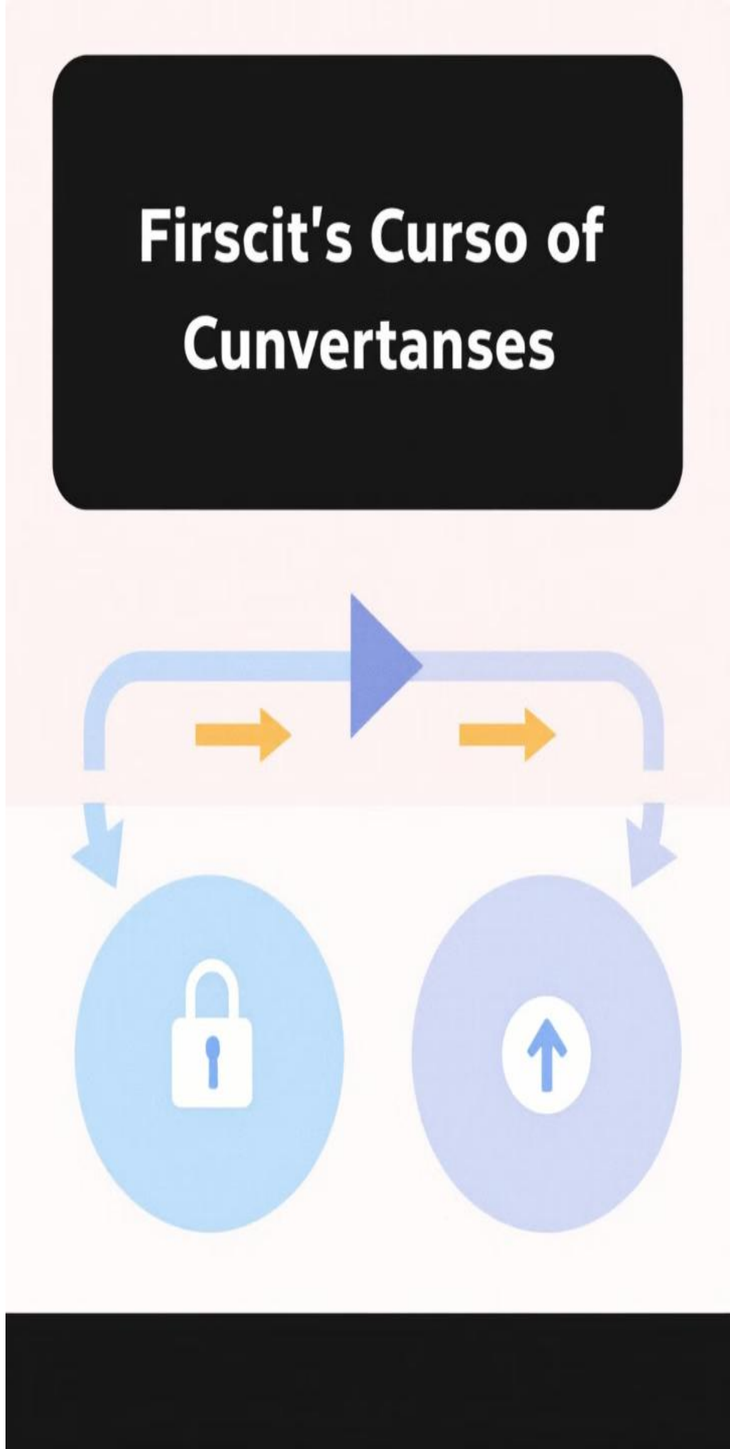
Tipos de desplazamiento y concurrencia

Desplazamiento (por defecto)

- **ResultSet.TYPE_FORWARD_ONLY**: El cursor solo puede avanzar hacia adelante.
- **TYPE_SCROLL_INSENSITIVE**: el cursor se mueve en ambas direcciones. Es una "foto" del momento de la consulta: no ve los cambios que otros hagan después en la base de datos.
- **TYPE_SCROLL_SENSITIVE**: se mueve en ambas direcciones y, en teoría, sí ve esos cambios. ⚠ MySQL Connector/J no lo admite y lo trata como INSENSITIVE.
- Métodos de navegación (solo con los tipos SCROLL): previous() fila anterior ; first() primera ; last() última ; absolute(n) fila n ; beforeFirst() / afterLast()
  Con TYPE_FORWARD_ONLY, llamar a cualquiera de ellos lanza una SQLException.

Concurrencia

- **ResultSet.CONCUR_READ_ONLY**: ResultSet de solo lectura.
- **ResultSet.CONCUR_UPDATABLE**: permite modificar la base de datos a través del ResultSet (con updateXXX() + updateRow()).



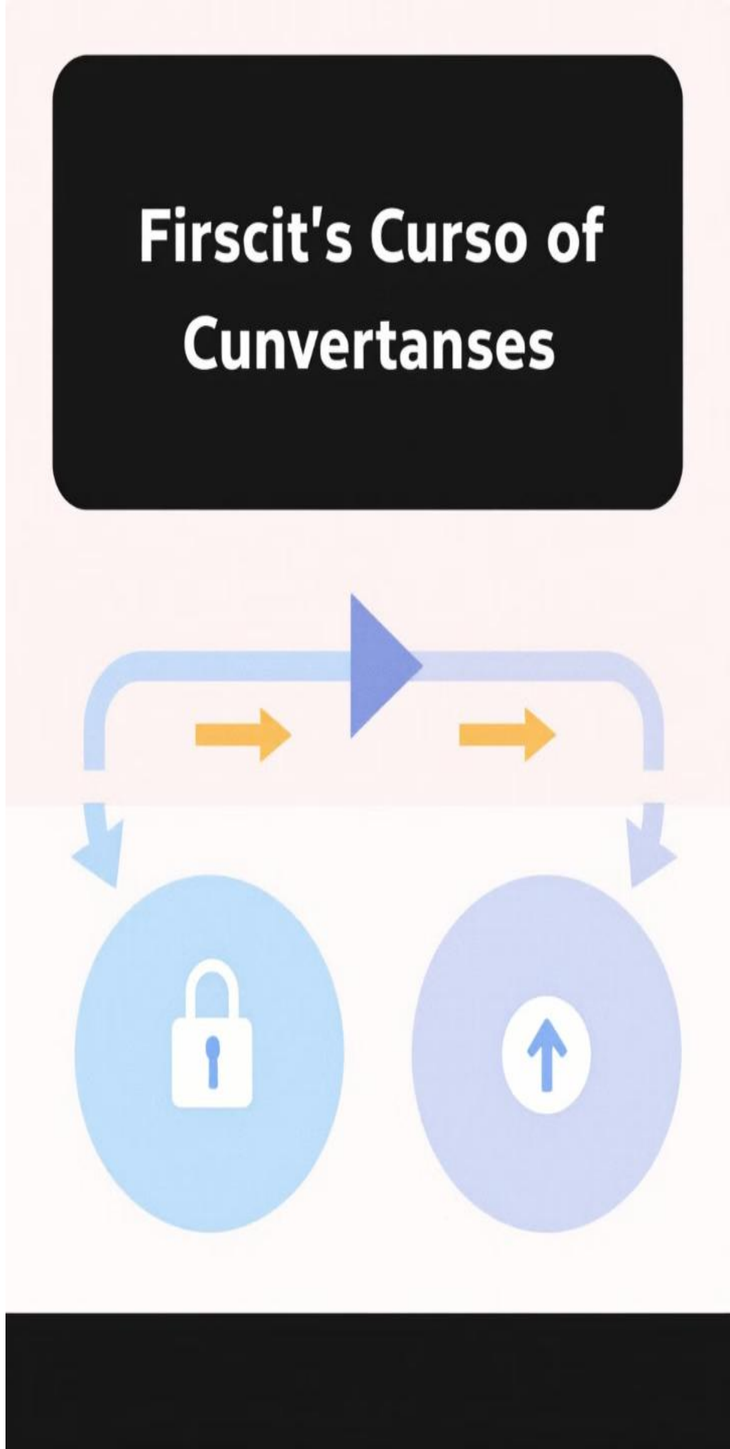
6.1. Tipos de ResultSet y su configuración

Tipos de desplazamiento y concurrencia

Ejemplo

Creación de un Statement para un ResultSet actualizable y bidireccional

```
Statement stmt = connection.createStatement(
    ResultSet.TYPE_SCROLL_INSENSITIVE,   // bidireccional
    ResultSet.CONCUR_UPDATABLE           // actualizable
);
```

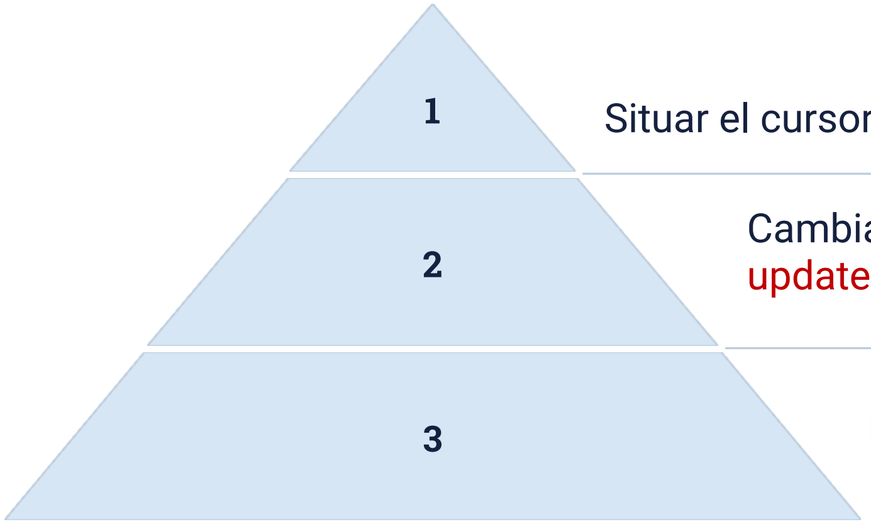


6.2. Modificación de datos con ResultSet actualizable

- Un ResultSet actualizable permite realizar **operaciones de modificación sobre las filas de datos de una tabla**.

- Es útil cuando el nuevo valor de cada fila **se decide en Java, fila a fila**: con lógica del programa, con datos que no están en la base de datos o con la confirmación del usuario. Por ejemplo, una pantalla que muestra los productos y deja editar el precio de cada uno.

¿Cómo se modifica una fila?



1. Situar el cursor en la fila (**next()**, **absolute(n)**…).
2. Cambiar el valor en el ResultSet: updateString(), updateInt(), updateBigDecimal()…
3. Enviar el cambio a la base de datos: updateRow().

También se puede borrar la fila actual con deleteRow() o insertar una nueva con moveToInsertRow() + updateXXX() + insertRow().
⚠ En MySQL, la consulta debe ser sobre una sola tabla, sin funciones y debe incluir la clave primaria. Si no, salta el error “Result Set not updatable”.

6.2. Modificación de datos con ResultSet actualizable

Ejemplo: modificar precios de una tabla

- El método modificarPrecios() recorre la tabla PRODUCTOS y multiplica el precio de cada producto por un factor (1.10 = subir un 10 %; 0.90 = bajar un 10 %).
- Basta con un ResultSet TYPE_FORWARD_ONLY (solo avanza) y CONCUR_UPDATABLE (permite modificar).

```java
public void modificarPrecios(Connection con, BigDecimal factor) throws SQLException {
   // Una sola tabla y con su clave primaria: requisito de MySQL
   String sql = "SELECT id, nombre, precio FROM PRODUCTOS";

   try (Statement stmt = con.createStatement(ResultSet.TYPE_FORWARD_ONLY,
         ResultSet.CONCUR_UPDATABLE);
       ResultSet rs = stmt.executeQuery(sql)) {

      while (rs.next()) {
         BigDecimal precio = rs.getBigDecimal("precio");          // 1. leer el valor actual
         BigDecimal nuevo = precio.multiply(factor)
               .setScale(2, RoundingMode.HALF_UP); //   calcular el nuevo (2 decimales)
         rs.updateBigDecimal("precio", nuevo);                    // 2. cambiarlo en el ResultSet
         rs.updateRow();                                  // 3. enviarlo a la base de datos
      }
   } // try-with-resources cierra ResultSet y Statement
}
```

```sql
CREATE TABLE PRODUCTOS (
    id     INT PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL,
    precio DECIMAL(8,2) NOT NULL
);
```

Modificación sobre tabla de base de datos con ResultSet actualizable

TAREA 06

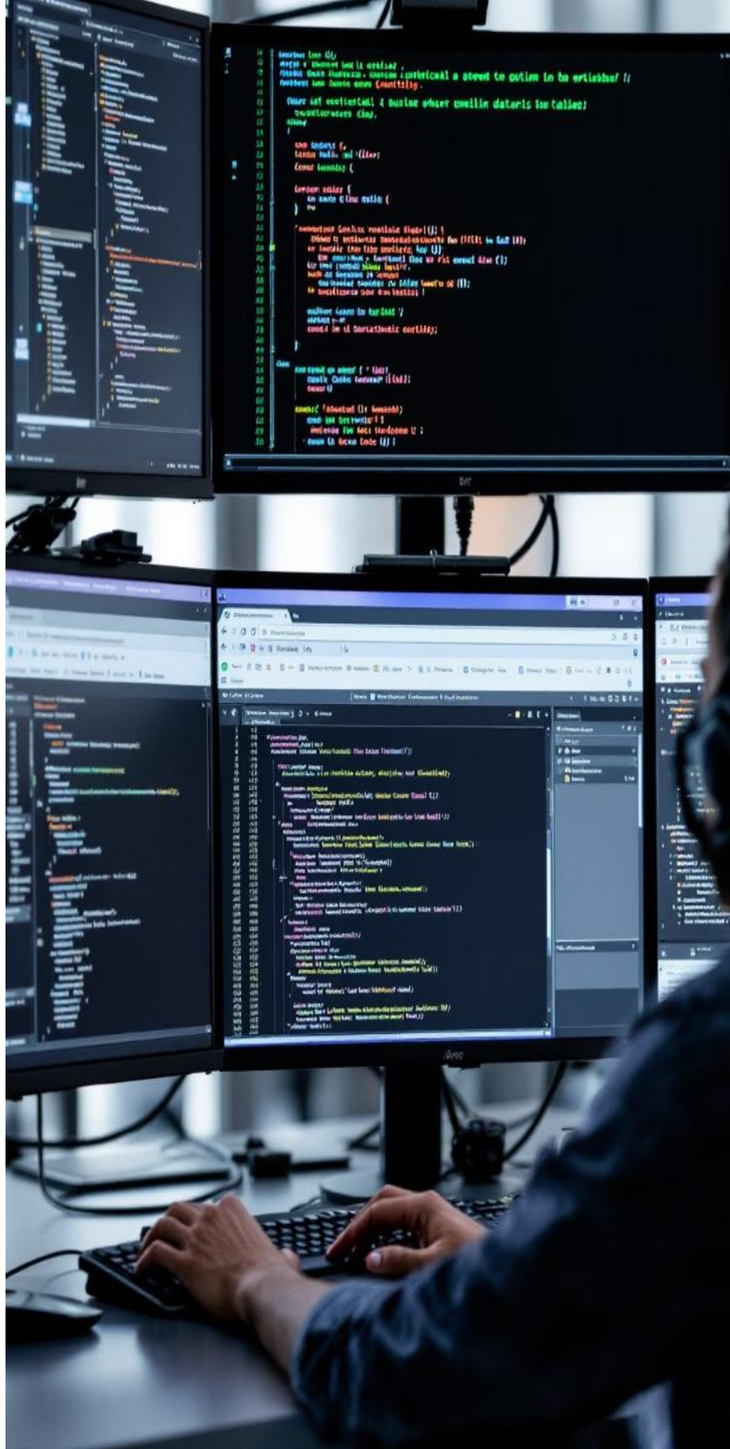
7. Ejecución de procedimientos almacenados

¿Qué son procedimientos almacenados?

- Los procedimientos almacenados son **bloques de código SQL** que pueden incluir lógica condicional, bucles y otras estructuras de control.

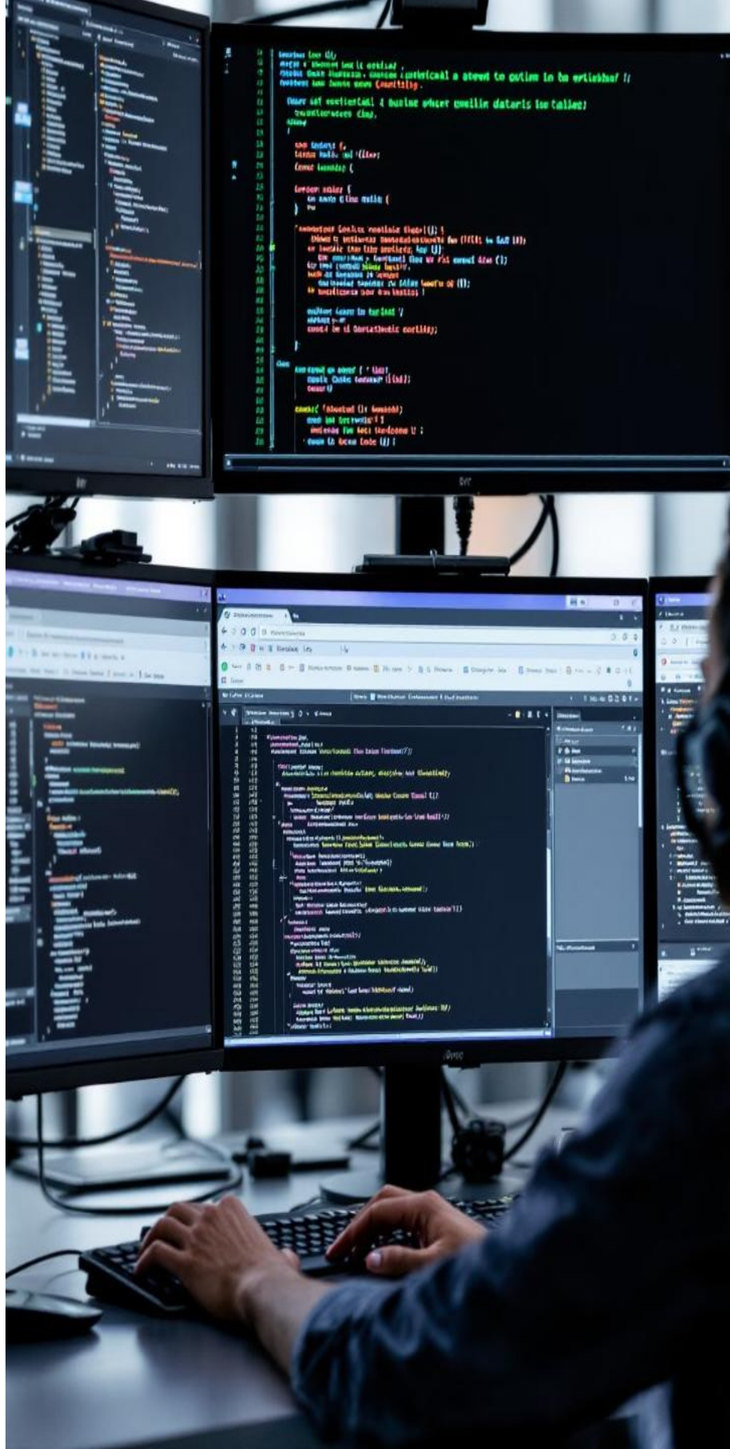
- Estos procedimientos **se ejecutan directamente en el sistema gestor de base de datos**, mejorando el rendimiento y la eficiencia.

- Se crean una sola vez (**CREATE PROCEDURE**) y quedan guardados en la base de datos. Desde Java no se escribe su código: solo se llaman por su nombre.



Características de procedimientos almacenados

- **Gestión de parámetros**
  - Pueden tener parámetros de entrada (IN), de salida (OUT) y de entrada/salida (INOUT).
  - Las funciones (**CREATE FUNCTION**) además devuelven un valor, como un método de Java con return.
- **Compatibilidad**
  - La mayoría de SGBD modernos, como MySQL, Oracle y SQL Server, soportan procedimientos almacenados.
  - Los gestores embebidos como SQLite no los admiten: esta parte de la unidad se trabaja solo con MySQL.



7.1. La interfaz CallableStatement

1 Interfaz CallableStatement

La interfaz **CallableStatement** en JDBC permite ejecutar procedimientos almacenados en la base de datos.

2 Creación de CallableStatement

Se crea con el método **prepareCall()** de la clase **Connection**, especificando el nombre del procedimiento y los parámetros necesarios, usando la sintaxis estándar de JDBC "**{call Nombre(?, ?)}**", con un ? por parámetro. El driver la traduce al CALL de cada gestor.

3 Asignación de parámetros

Se utilizan métodos como **setString()** para asignar valores a los parámetros de entrada del procedimiento.

4 Ejecución del procedimiento, según lo que haga

- Devuelve filas (contiene un SELECT) → executeQuery() → ResultSet.
- Modifica datos → executeUpdate() → filas afectadas.
- Tiene parámetros de salida, o no sabes qué devuelve → execute().



Ejemplo 1 de llamada a un procedimiento almacenado

Existe un procedimiento almacenado llamado **DameAlbumes** que recibe dos parámetros de entrada (**titulo y autor**) y devuelve todos los álbumes cuyo título y autor coincidan con los valores dados.
La llamada se realiza con CallableStatement de la siguiente forma:

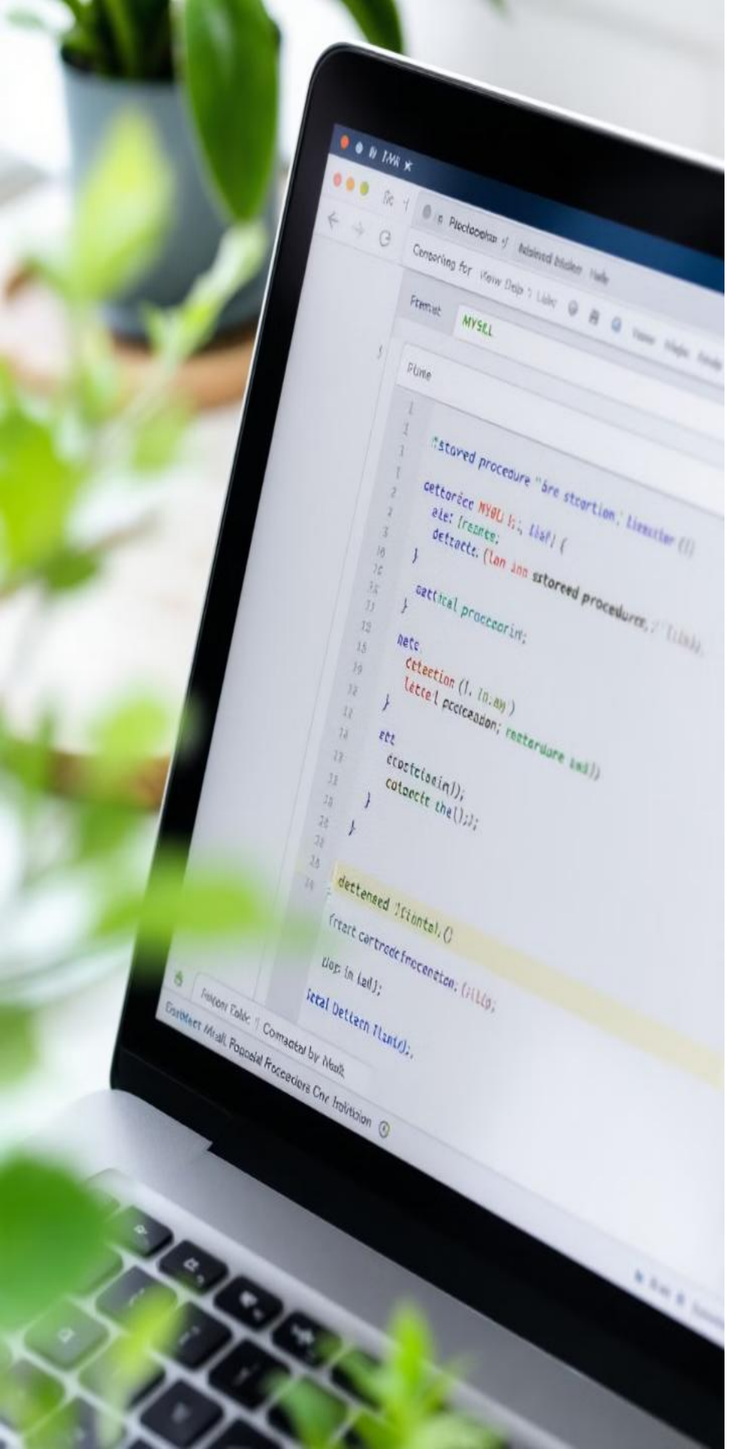
```
CallableStatement callableStatement = connection.prepareCall("{call DameAlbumes(?, ?)}");

// Asignar valores a los parámetros de entrada
callableStatement.setString(1, "Título del álbum");
callableStatement.setString(2, "Nombre del autor");

// Ejecutar el procedimiento
ResultSet resultSet = callableStatement.executeQuery();

// Procesar los resultados
while (resultSet.next()) {
    System.out.println("Álbum: " + resultSet.getString("nombre_album"));
}
```





Ejemplo 2 de llamada a un procedimiento almacenado

Cada SGBD tiene su propia sintaxis para definir estos procedimientos.

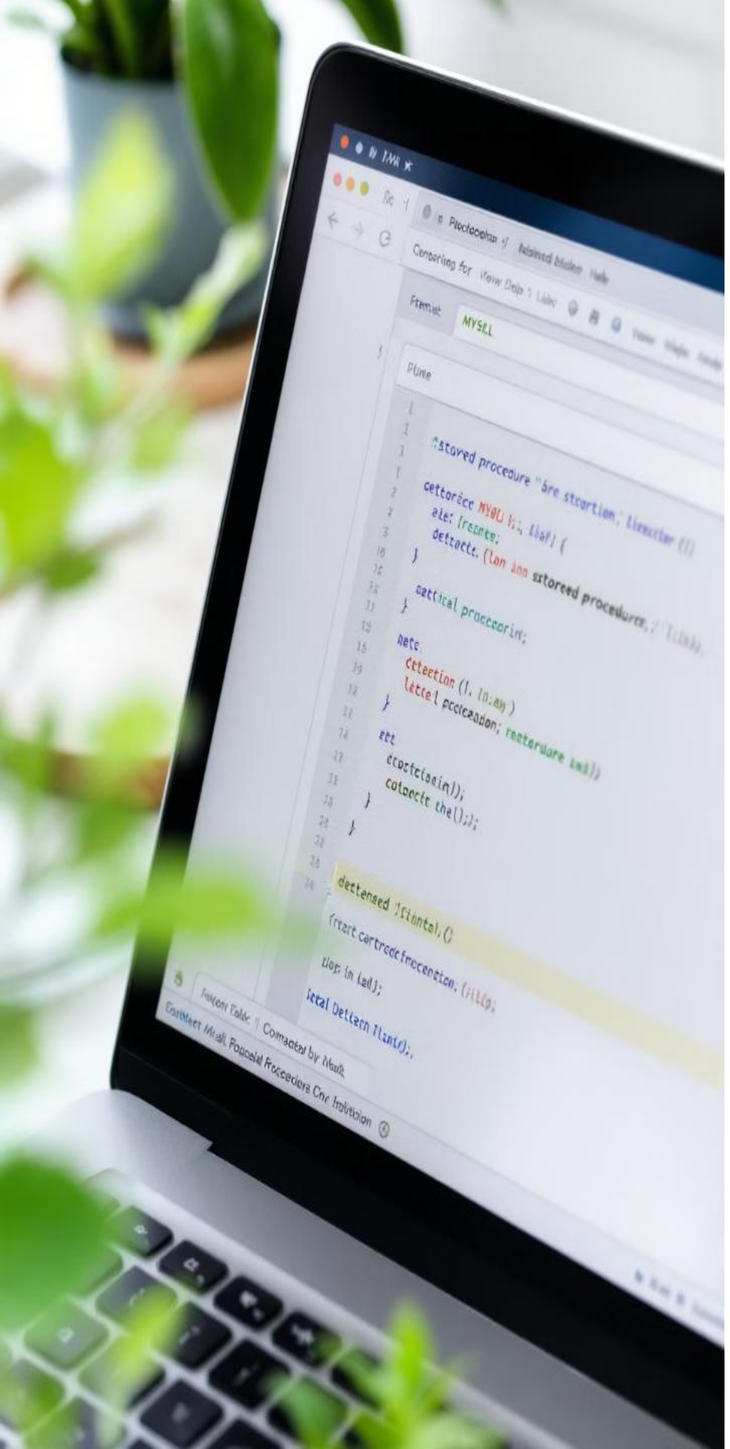
1

Definir procedimiento

La llamada desde JDBC es igual en todos los gestores, pero la definición usa la sintaxis propia de cada uno. Esta es la de MySQL.

```
DELIMITER //
CREATE PROCEDURE ActualizarPrecio(IN productId INT, IN porcentaje FLOAT)
BEGIN
    UPDATE Productos SET precio = precio * porcentaje WHERE id = productId;
END //
DELIMITER ;
```

- Este ejemplo crea un procedimiento llamado **ActualizarPrecio** que ajusta el precio de un producto en función de un porcentaje.
- Este procedimiento recibe dos parámetros: el ID del producto y el porcentaje de ajuste.



Ejemplo 2 de llamada a un procedimiento almacenado

2

Llamada desde JDBC

- La forma de llamar a los procedimientos almacenados en JDBC es uniforme, independientemente del SGBD.
- Se utiliza **CallableStatement** para preparar y ejecutar la llamada.

```java
import java.sql.CallableStatement;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class EjecutarProcedimiento {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/nombre_base_datos";
        String user = "usuario";
        String password = "contrasena";

        try (Connection connection = DriverManager.getConnection(url, user, password)) {
            // Preparar la llamada al procedimiento almacenado
            CallableStatement callableStatement = connection.prepareCall("{call ActualizarPrecio(?, ?)}");

            // Asignar valores a los parámetros de entrada
            callableStatement.setInt(1, 101); // ID del producto
            callableStatement.setFloat(2, 1.10f); // Ajuste del 10%

            // Ejecutar el procedimiento
            int filasAfectadas = callableStatement.executeUpdate();
            System.out.println("Filas afectadas: " + filasAfectadas);

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Explicación del Ejemplo

1

Preparación: prepareCall("{call ActualizarPrecio(?, ?)}")

Prepara la llamada al procedimiento **ActualizarPrecio** con dos parámetros de entrada.

2

Asignación: setInt(1, 101) y setFloat(2, 1.10f)

Asigna el ID del producto y el porcentaje de ajuste al precio.

3

Ejecución: executeUpdate()

Ejecuta el procedimiento y devuelve el número de filas afectadas.

Ventajas de usar procedimientos almacenados



Rendimiento

Reduce la transferencia de datos y optimiza las operaciones al procesar en el servidor.



Reutilización

Los procedimientos almacenados pueden reutilizarse en múltiples aplicaciones y consultas.



Seguridad

Limita las interacciones directas con las tablas y permite controles de acceso más específicos.

Inconvenientes de usar procedimientos almacenados

Dependen del gestor

Cada uno tiene su sintaxis. Si se cambia de MySQL a Oracle, hay que reescribirlos, y SQLite ni siquiera los admite.

La lógica queda repartida

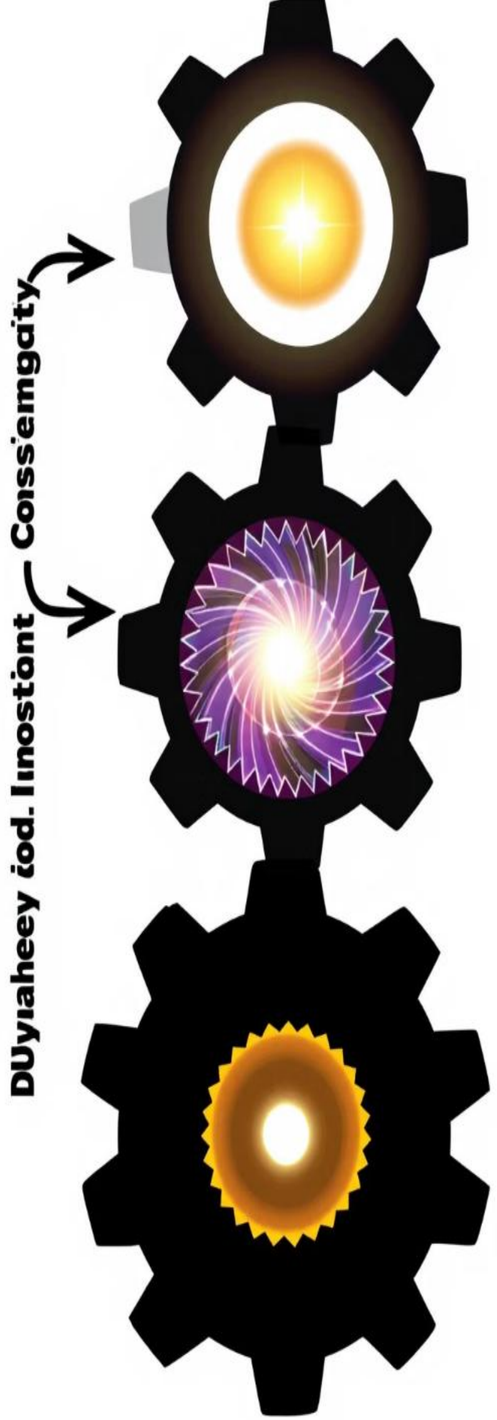
Queda repartida entre Java y la base de datos, lo que la hace más difícil de depurar, probar y versionar con Git.

Importación y consultas con procedimientos almacenados

TAREA 07

8. Gestión de transacciones

Las transacciones en bases de datos son un **conjunto de operaciones SQL** que **se ejecutan como una sola unidad lógica de trabajo**, asegurando la consistencia e integridad de los datos.





Características ACID de las transacciones

1 Atomicidad

Todas las operaciones de la transacción se completan en su totalidad o no se realiza ninguna. Si alguna falla, todas se cancelan.

2 Consistencia

La transacción lleva la base de datos de un estado válido a otro, cumpliendo todas las reglas de integridad.

3 Aislamiento

La transacción se ejecuta de manera aislada de otras transacciones en proceso, evitando interferencias.

4 Durabilidad

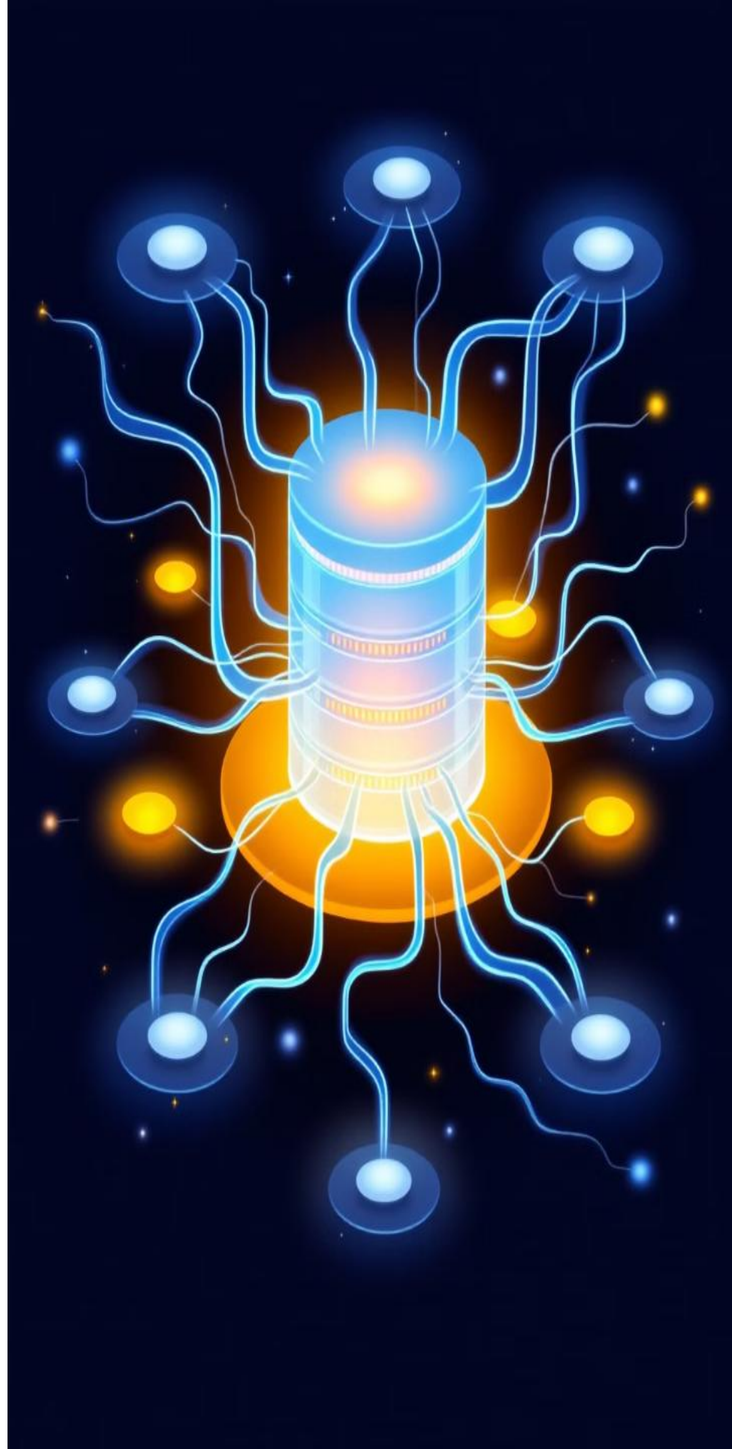
Una vez confirmada, la transacción es permanente, incluso en caso de fallos del sistema.

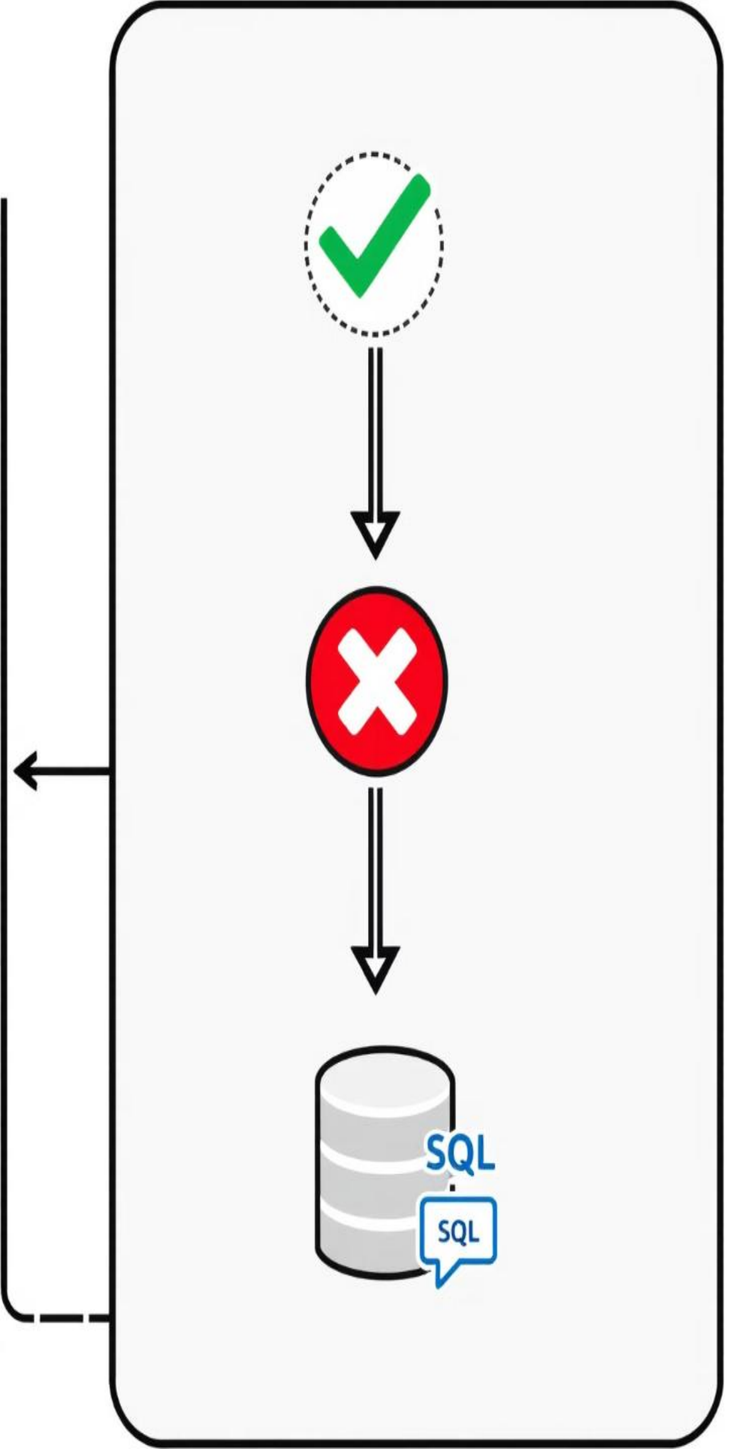
Implementación de transacciones en JDBC

La gestión de transacciones en JDBC depende de la configuración de **auto-commit**.

Por defecto, cada sentencia SQL se confirma automáticamente en la base de datos.

Para ejecutar varias operaciones SQL como una sola transacción, se debe desactivar el auto-commit.





Pasos para gestionar una transacción en JDBC

1

Desactivar Auto-commit

Se desactiva el modo auto-commit mediante **setAutoCommit(false)**, permitiendo que las operaciones SQL no se confirmen automáticamente.

2

Ejecutar las operaciones SQL

Ejecuta todas las operaciones de la transacción (por ejemplo, varias sentencias **INSERT** o **UPDATE**).

3

Confirmar o deshacer los cambios

Si todas las operaciones se ejecutan correctamente, se llama a **commit()** para confirmar la transacción. Si ocurre un error, se ejecuta **rollback()** para deshacer los cambios.



Ejemplo de transacción en JDBC

1

Inserción de registros

Insertar dos registros en una tabla de álbumes.

2

Confirmación de Transacción

Si ambas operaciones **INSERT** se ejecutan sin problemas, **confirmar la transacción** con `commit()`.

3

Deshacer Cambios

Si ocurre un **error**, **deshacer los cambios** con `rollback()`.



Ejemplo de transacción en JDBC

```java
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.sql.Statement;

public class GestionTransacciones {

    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/loren";
        String user = "loren";
        String password = "loren";

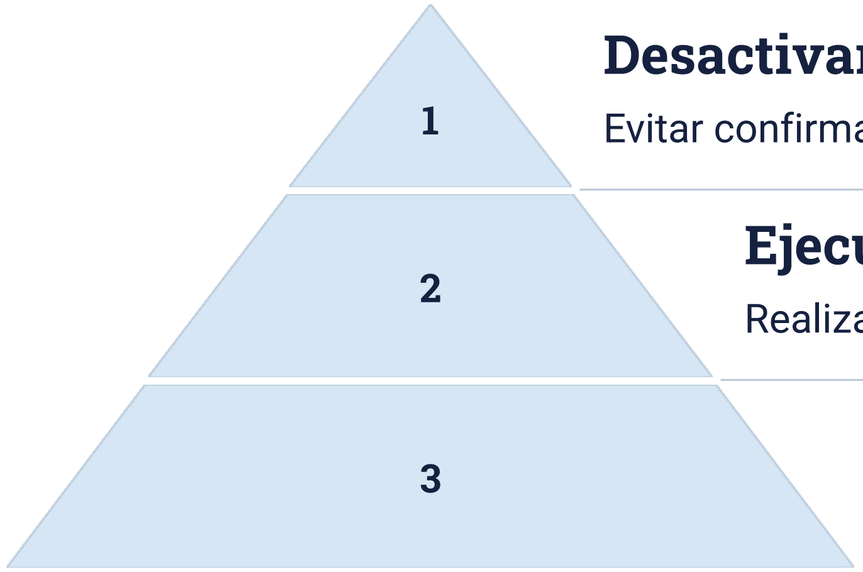
        try (Connection connection = DriverManager.getConnection(url, user, password)) {
            // Desactivar auto-commit para iniciar la transacción
            connection.setAutoCommit(false);

            try (Statement statement = connection.createStatement()) {
                // Ejecutar operaciones como parte de una transacción
                statement.executeUpdate("INSERT INTO album (titulo, artista) VALUES ('Álbum1', 'Artista1')");
                statement.executeUpdate("INSERT INTO album (titulo, artista) VALUES ('Álbum2', 'Artista2')");

                // Confirmar la transacción
                connection.commit();
                System.out.println("Transacción completada con éxito.");
            } catch (SQLException e) {
                // Si ocurre un error, deshacer los cambios
                System.err.println("Error en la transacción. Realizando rollback.");
                connection.rollback();
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Explicación del código del ejemplo

El código explicado muestra los pasos clave para gestionar una transacción en JDBC: desactivar el modo de auto-confirmación, ejecutar las operaciones como parte de la transacción y, finalmente, confirmar los cambios o revertirlos en caso de que ocurra una excepción.



Desactivar auto-commit

Evitar confirmación automática de cada sentencia.

Ejecutar operaciones

Realizar sentencias INSERT como parte de la transacción.

Confirmar o revertir

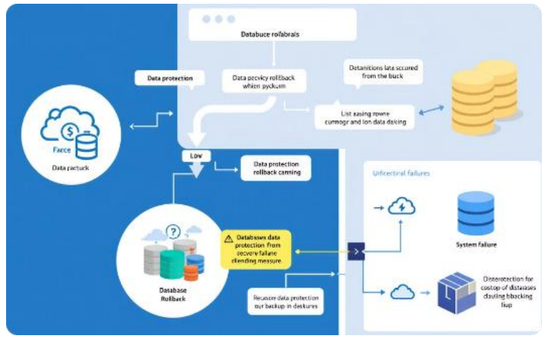
Confirmar cambios o revertir en caso de excepción.

Importancia de las transacciones



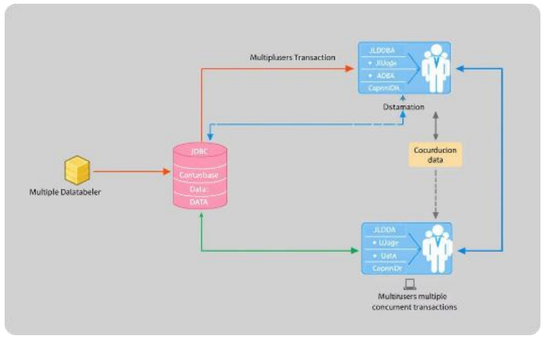
Garantía de integridad

Las transacciones aseguran que las operaciones críticas en la base de datos se completen correctamente, preservando la integridad de los datos.



Recuperación ante fallos

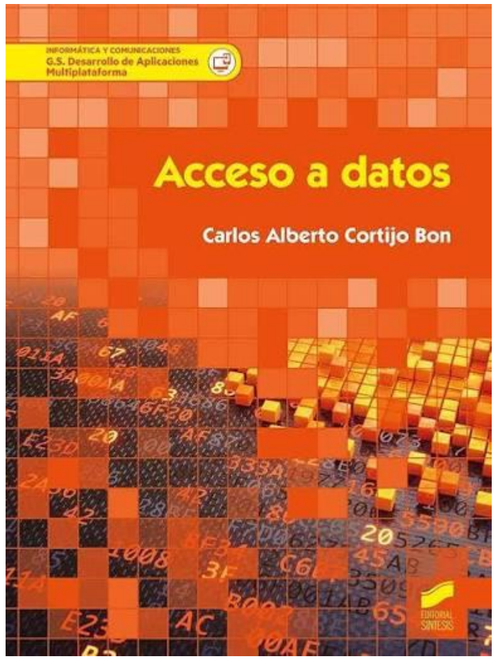
En caso de fallo, el método **rollback()** permite deshacer los cambios y mantener la consistencia de la información.



Desarrollo seguro

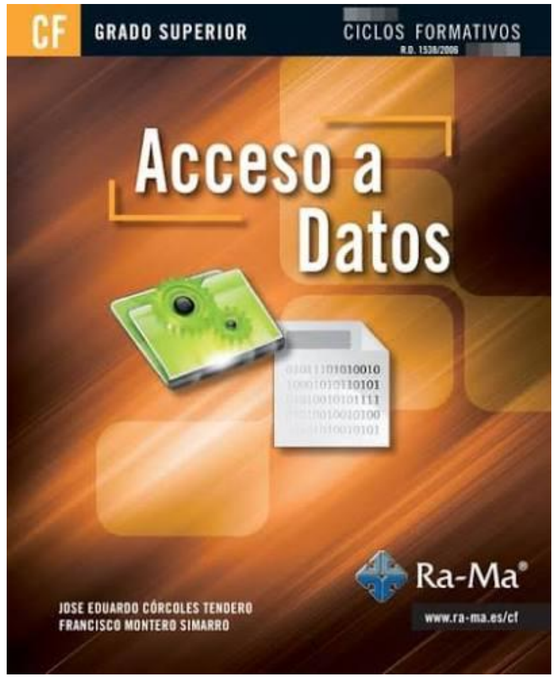
La estructura de transacción en JDBC facilita el desarrollo de aplicaciones confiables, independientemente de la base de datos utilizada.

Bibliografía



Cortijo Bon, Carlos A.

”Acceso a datos”. En editorial Síntesis (España). ISBN: 978-84-9171-356-2.



Córcoles Tendero, Jose Eduardo.

”Acceso a datos”. En editorial RA-MA (España). E-ISBN: 9788499643908



Camarena Estruch, Joan Gerard; Múrcia Andres, Jose A.

“Acceso a datos”. En editorial McGraw Hill. ISBN 9788448626730.